In [56]:
```python
import igraph as ig
import networkx as nx
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress
from collections import Counter
```

Q1 Generating Random Networks

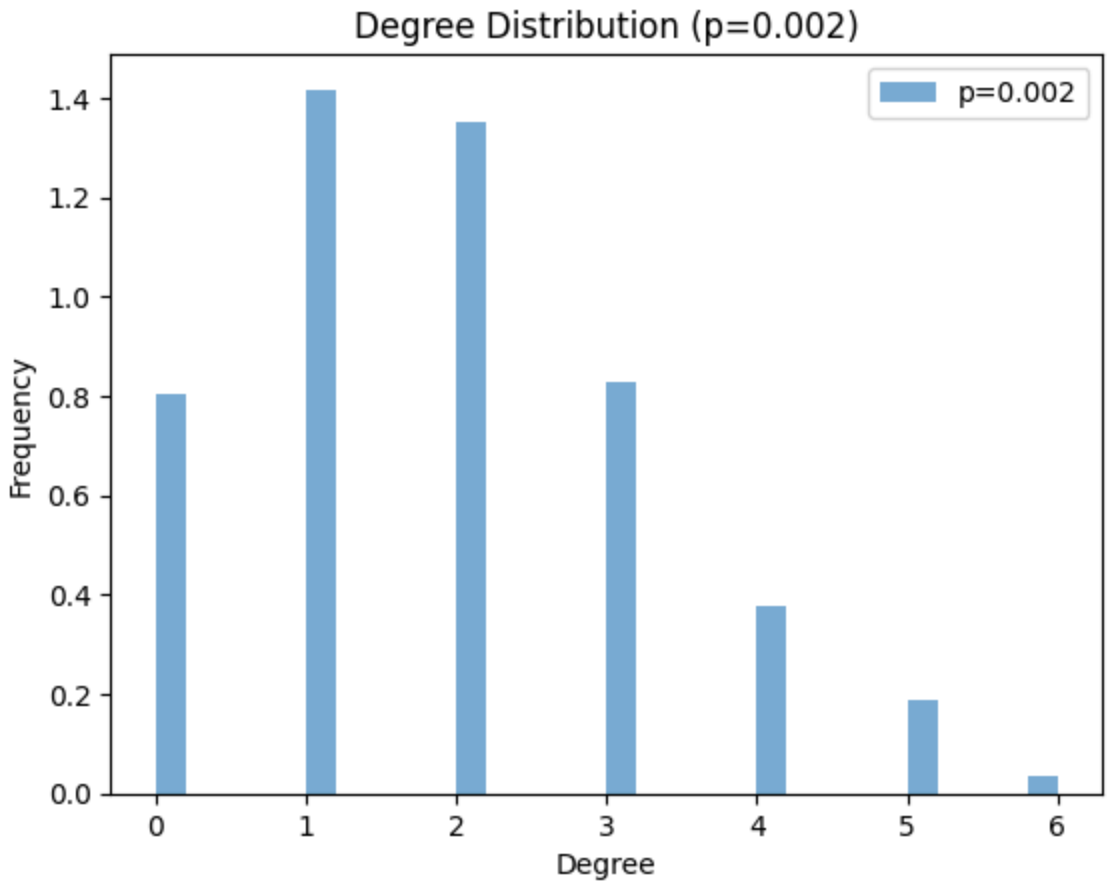
Q1.1 Create random networks using Erd¨os-R´enyi (ER) model

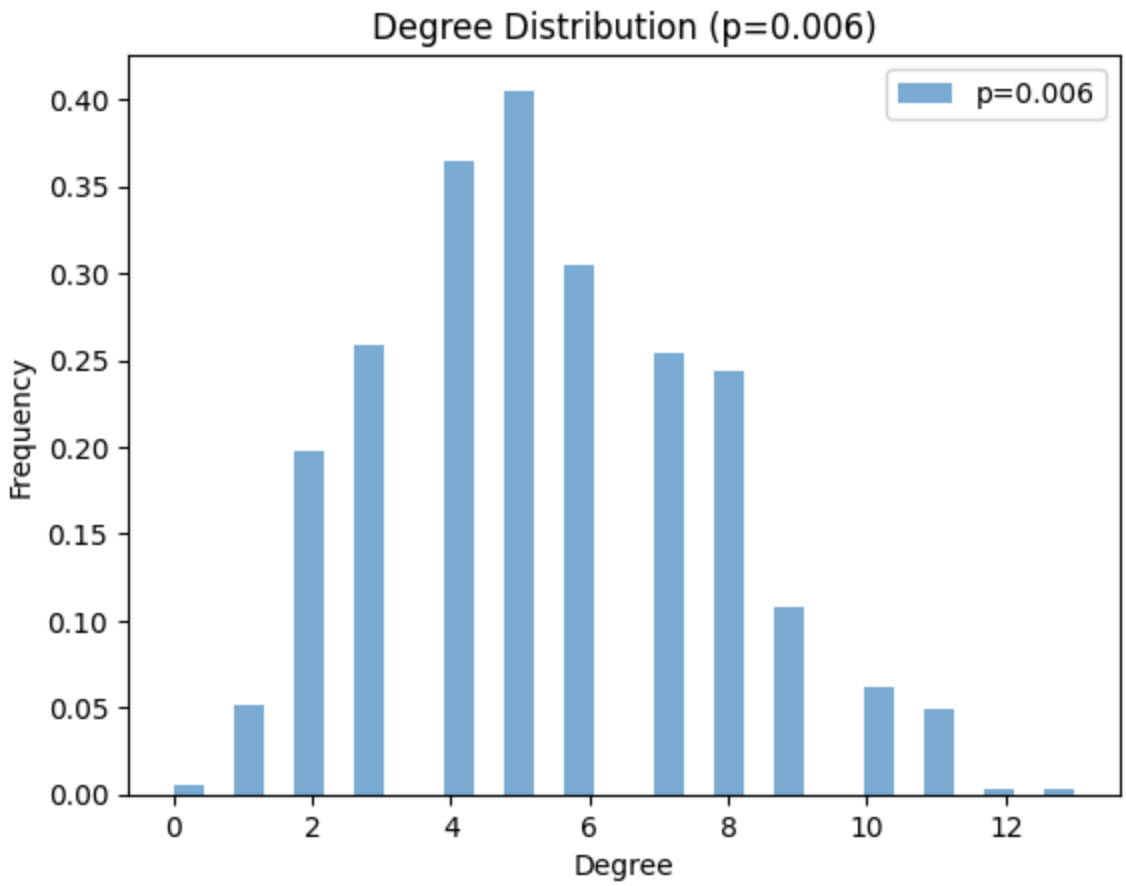
Q1.1(a)

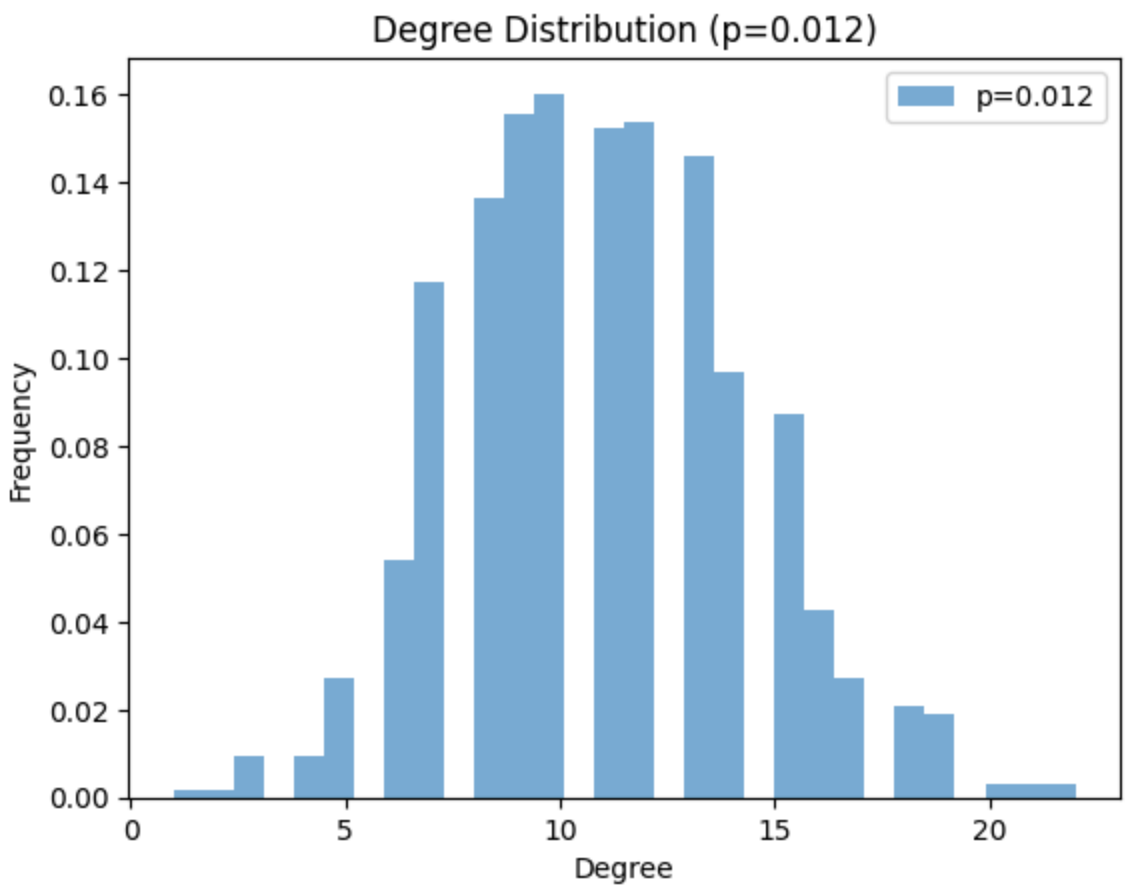
Create undirected random networks with n = 900 nodes, and the probability p for drawing an edge between two arbitrary vertices 0.002, 0.006, 0.012, 0.045, and 0.1. Plot the degree distributions. What distribution (linear/exponential/gaussian/binomial or something else) is observed? Explain why. Also, report the mean and variance of the degree distributions and compare them to the theoretical values.

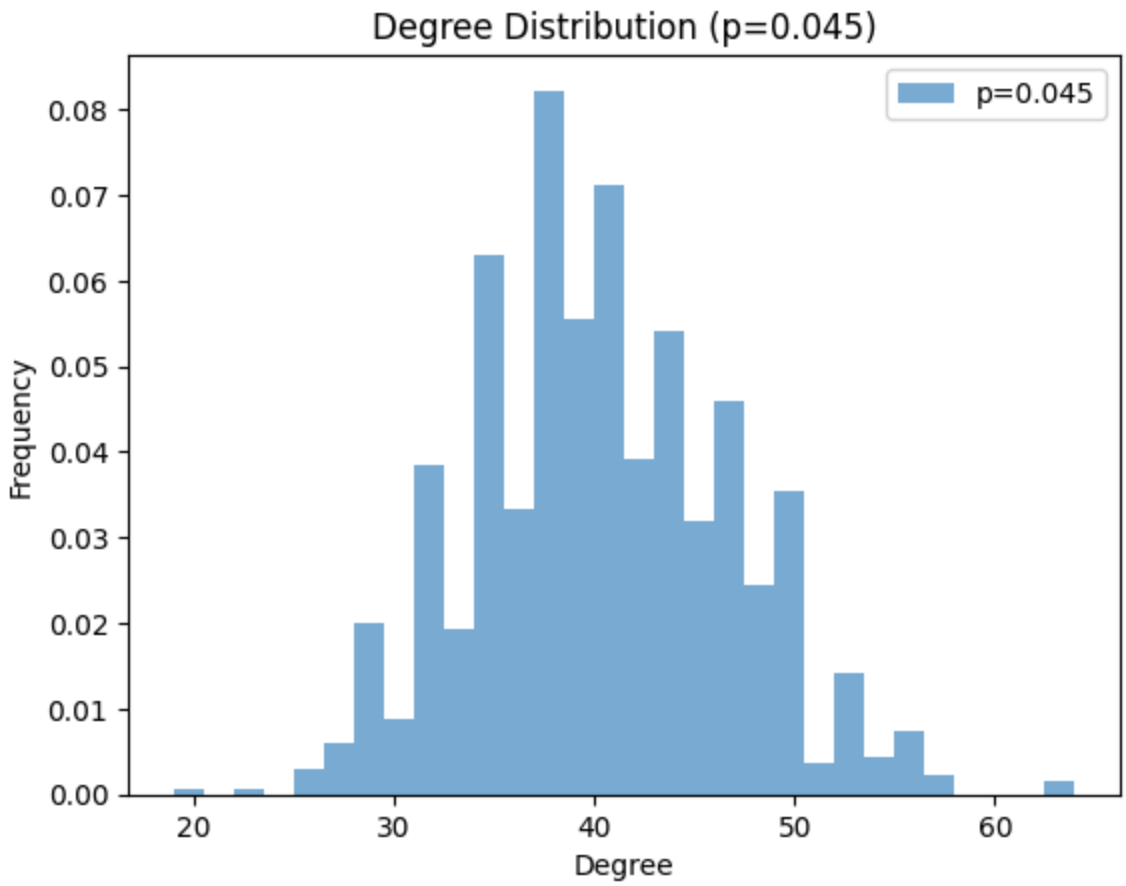
In [16]:
```python
n= 900
p_values = [0.002, 0.006, 0.012, 0.045, 0.1]

# Initialize results
results = {}

# Generate graphs and analyze degree distributions
for p in p_values:
    G = ig.Graph.Erdos_Renyi(n, p, directed=False)
    degrees = G.degree()

    # dist  = G.degree_distribution()

    # Compute mean and variance
    mean_degree = np.mean(degrees)
    var_degree = np.var(degrees)

    # Theoretical values
    mean_theoretical = p * (n - 1)
    var_theoretical = p * (1 - p) * (n - 1)

    results[p] = {
        "mean": mean_degree,
        "variance": var_degree,
        "mean_theoretical": mean_theoretical,
        "variance_theoretical": var_theoretical
    }
```

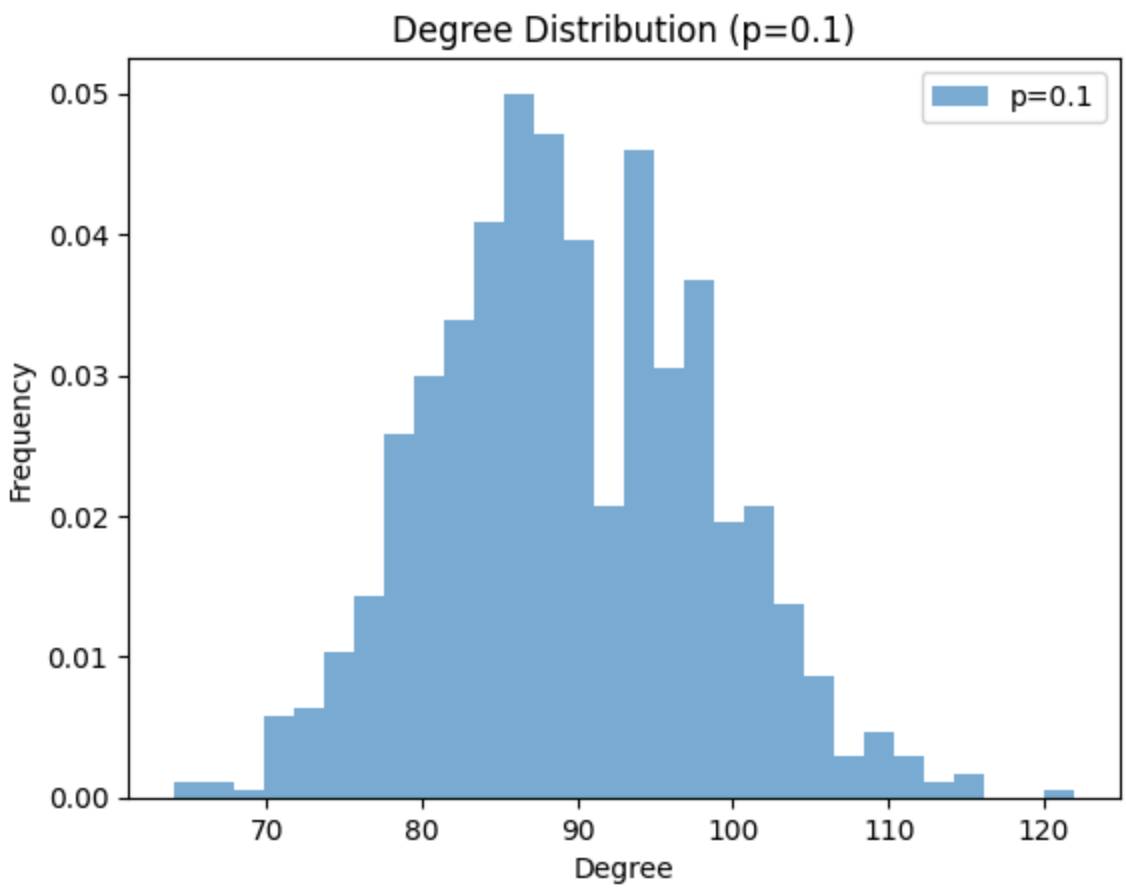
```python
    # Plot degree distribution
    plt.hist(degrees, bins=30, density=True, alpha=0.6, label=f"p={p}")
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.title(f"Degree Distribution (p={p})")
    plt.legend()
    plt.show()

# Print results
for p, stats in results.items():
    print(f"p={p}: Mean={stats['mean']:.2f}, Variance={stats['variance']:.2f}, "
          f"Theoretical Mean={stats['mean_theoretical']:.2f}, "
          f"Theoretical Variance={stats['variance_theoretical']:.2f}\n")
```




Degree Distribution (p=0.006)
p=0.006
Frequency
0.40
0.35
0.30
0.25
0.20
0.15
0.10
0.05
0.00
0
2
4
6
8
10
12
Degree


Degree Distribution (p=0.012)
p=0.012
Frequency
0.16
0.14
0.12
0.10
0.08
0.06
0.04
0.02
0.00
0
5
10
15
20
Degree


Degree Distribution (p=0.045)
p=0.045
Frequency
0.00
0.01
0.02
0.03
0.04
0.05
0.06
0.07
0.08
20
30
40
50
60
Degree


Degree Distribution (p=0.1)
p=0.1
Frequency
0.00
0.01
0.02
0.03
0.04
0.05
70
80
90
100
110
120
Degree

```
p=0.002: Mean=1.85, Variance=1.82, Theoretical Mean=1.80, Theoretical Variance=1.79

p=0.006: Mean=5.39, Variance=5.45, Theoretical Mean=5.39, Theoretical Variance=5.36

p=0.012: Mean=10.93, Variance=11.06, Theoretical Mean=10.79, Theoretical Variance=1
0.66

p=0.045: Mean=40.13, Variance=40.95, Theoretical Mean=40.45, Theoretical Variance=3
8.63

p=0.1: Mean=89.54, Variance=76.58, Theoretical Mean=89.90, Theoretical Variance=80.9
1
```

Q1.1(b)

For each p and n = 900, answer the following questions: Are all random realizations of the ER network connected? Numerically estimate the probability that a generated network is connected. For one instance of the networks with that p, find the giant connected component (GCC) if not connected. What is the diameter of the GCC?

In [ ]:

```python
n = 900  # Number of nodes
p_values = [0.002, 0.006, 0.012, 0.045, 0.1]  # Edge probabilities
num_trials = 100
results = {}


for p in p_values:
    connected_count= 0
    for _ in range(num_trials):
        g = ig.Graph.Erdos_Renyi(n, p)
        if g.is_connected():
            connected_count += 1

    connectivity_prob = connected_count / num_trials

    g = ig.Graph.Erdos_Renyi(n, p, directed=False)

    if g.is_connected():
        gcc_diameter = g.diameter()
        print(f"p={p}: Network is connected. Diameter={gcc_diameter}")
    else:
        components = g.components()
        gcc = components.giant()
        gcc_diameter = gcc.diameter()
        print(f"p={p}: Network is NOT connected. Diameter of GCC={gcc_diameter}")

    results[p] = {
        "connectivity_probability": connectivity_prob,
        "gcc_diameter": gcc_diameter
    }

print("\nEstimated Connectivity Probabilities:")
```

```
for p, data in results.items():
    print(f"p={p}: Connectivity Probability={data['connectivity_probability']:.2f},
```

```
p=0.002: Network is NOT connected. Diameter of GCC=31
p=0.006: Network is NOT connected. Diameter of GCC=8
p=0.012: Network is connected. Diameter=5
p=0.045: Network is connected. Diameter=3
p=0.1: Network is connected. Diameter=3

Estimated Connectivity Probabilities:
p=0.002: Connectivity Probability=0.00, GCC Diameter=31
p=0.006: Connectivity Probability=0.04, GCC Diameter=8
p=0.012: Connectivity Probability=0.98, GCC Diameter=5
p=0.045: Connectivity Probability=1.00, GCC Diameter=3
p=0.1: Connectivity Probability=1.00, GCC Diameter=3
```

Q1.1(c)

It turns out that the normalized GCC size (i.e., the size of the GCC as a fraction of the total network size) is a highly nonlinear function of p, with interesting properties occurring for values where p = O(1 n) and p = O(ln n n).

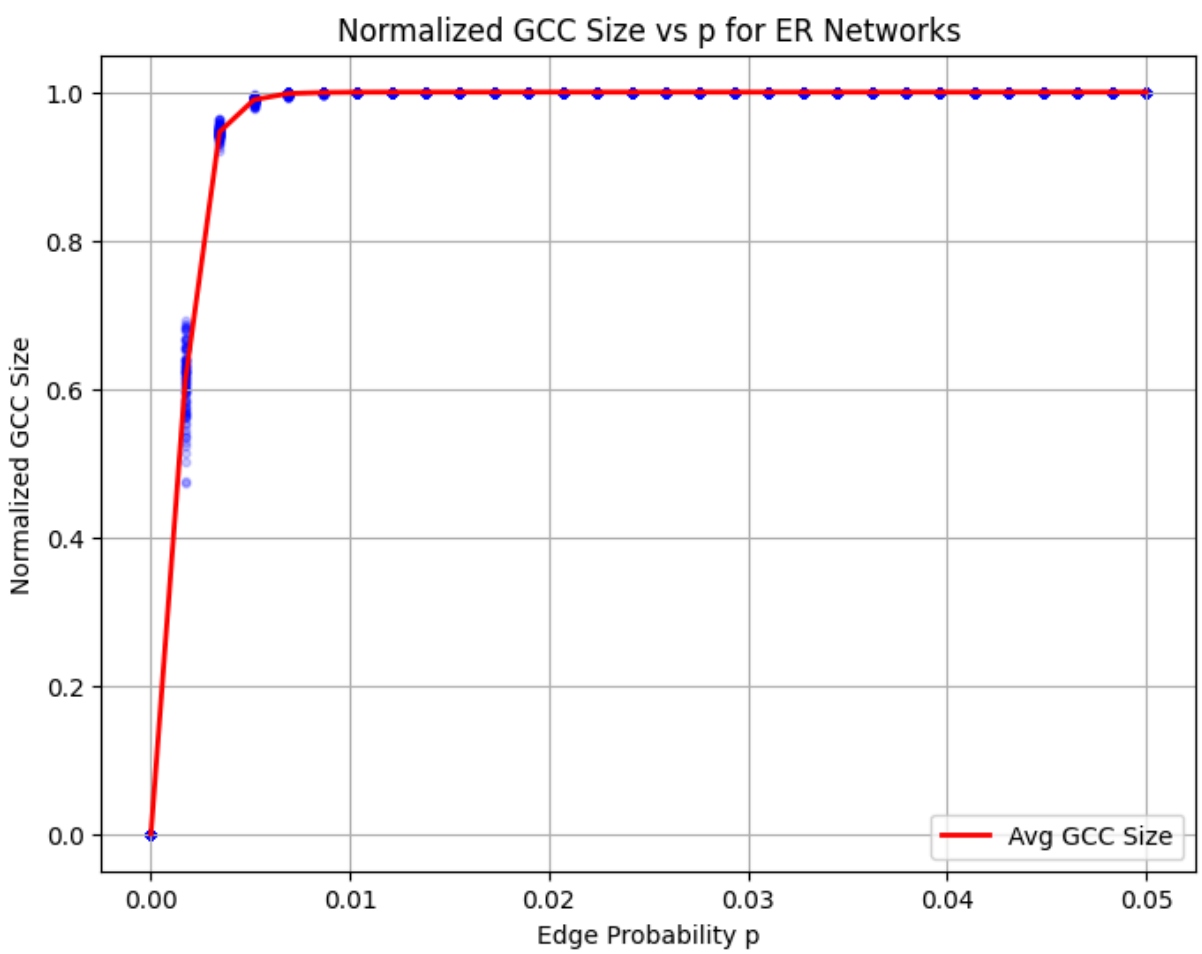
For n = 900, sweep over values of p from 0 to a pmax that makes the network almost surely connected and create 100 random networks for each p. pmax should be roughly determined by yourself. Then scatter plot the normalized GCC sizes vs p. Plot a line of the average normalized GCC sizes for each p along with the scatter plot.

1. Empirically estimate the value of p where a giant connected component starts to emerge (define your criterion of “emergence”)? Do they match with theoretical values mentioned or derived in lectures?
2. Empirically estimate the value of p where the giant connected component takes up over 99% of the nodes in almost every experiment.

In [ ]:

```
n = 900  # Number of nodes
# p_values = [0.002, 0.006, 0.012, 0.045, 0.1]  # Edge probabilities
p_values = np.linspace(0, 0.05, 30)  # Sweep p from 0 to 0.05 with 30 points
num_trials = 100
results = {}
gcc_sizes = {p: [] for p in p_values}

for p in p_values:
    connected_count= 0
    for _ in range(num_trials):
        g = ig.Graph.Erdos_Renyi(n, p)
        gcc = g.components().giant()
        gcc_size = gcc.vcount()/ g.vcount()  # Normalize by total number of nodes
        gcc_sizes[p].append(gcc_size)

# Compute average GCC sizes
avg_gcc_sizes = [np.mean(gcc_sizes[p]) for p in p_values]

# Scatter plot
```

```python
plt.figure(figsize=(8, 6))
for p in p_values:
    plt.scatter([p] * num_trials, gcc_sizes[p], color='blue', alpha=0.2, s=10)

# Plot average GCC size
plt.plot(p_values, avg_gcc_sizes, color='red', label="Avg GCC Size", linewidth=2)

plt.xlabel("Edge Probability p")
plt.ylabel("Normalized GCC Size")
plt.title("Normalized GCC Size vs p for ER Networks")
plt.legend()
plt.grid()
plt.show()

# Finding emergence point
threshold = 0.1  # Define emergence as GCC covering >10% of nodes
p_emergence = next(p for p in p_values if np.mean(gcc_sizes[p]) > threshold)
print(f"Giant component emerges at approximately p ≈ {p_emergence:.4f}")

# Finding full connectivity point
threshold_99 = 0.99  # Define full coverage as GCC >99% in nearly all trials
p_full = next(p for p in p_values if np.mean(gcc_sizes[p]) > threshold_99)
print(f"GCC covers >99% of nodes at approximately p ≈ {p_full:.4f}")
```



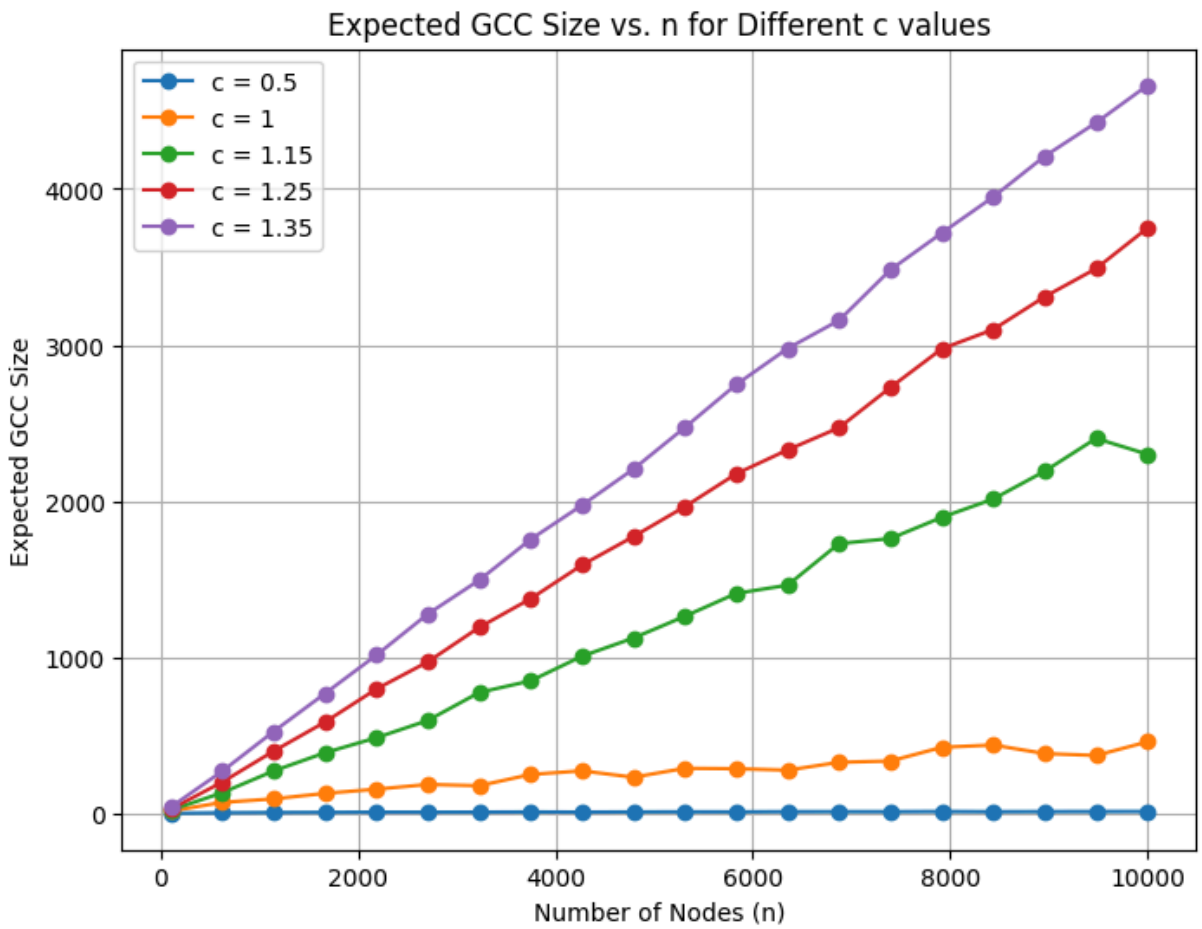

```
Giant component emerges at approximately p ≈ 0.0017
GCC covers >99% of nodes at approximately p ≈ 0.0069
```

Q1.1(d)

1. Define the average degree of nodes c = n × p = 0.5. Sweep over the number of nodes, n, ranging from 100 to 10000. Plot the expected size of the GCC of ER networks with n nodes and edge-formation probabilities p = c/n, as a function of n. What trend is observed?
2. Repeat the same for c = 1.
3. Repeat the same for values of c = 1.15, 1.25, 1.35, and show the results for these three values in a single plot.
4. What is the relation between the expected GCC size and n in each case?

In [33]:
```
n_values = np.linspace(100, 10000, 20, dtype=int)  # Sweep n from 100 to 10000
c_values = [0.5, 1, 1.15, 1.25, 1.35]  # Different c values
num_trials = 50  # Number of random realizations per n

gcc_sizes = {c: [] for c in c_values}

# Loop over different c values
for c in c_values:
    print(f"Processing c = {c}...")

    # Compute GCC size for each n
    for n in n_values:
        p = c / n  # Edge probability
        avg_gcc_size = 0

        # Run multiple realizations for averaging
        for _ in range(num_trials):
            g = ig.Graph.Erdos_Renyi(n, p)
            if g.is_connected():
                gcc_size = n
            else:
                gcc = g.components().giant()
                gcc_size = gcc.vcount()
            avg_gcc_size += gcc_size
        avg_gcc_size /= num_trials  # Compute the average
        gcc_sizes[c].append(avg_gcc_size)

# Plot results
plt.figure(figsize=(8,6))
for c in c_values:
    plt.plot(n_values, gcc_sizes[c], marker='o', label=f'c = {c}')

plt.xlabel("Number of Nodes (n)")
plt.ylabel("Expected GCC Size")
plt.title("Expected GCC Size vs. n for Different c values")
plt.legend()
plt.grid()
plt.show()
```

```
Processing c = 0.5...
Processing c = 1...
Processing c = 1.15...
Processing c = 1.25...
Processing c = 1.35...
```



1. For c = 0.5 the GCC size remains small even as the number of nodes increases
2. For c = 1 the GCC also remains small as the number of nodes increases, although it is larger than c=0.5, we can see a GCC beginning to appear
3. For c = 1.15 onwards the GCC grows linearly with the increase in number of nodes n

Q2 Create networks using preferential attachment model

Q1.2(a)

Create an undirected network with n = 1050 nodes, with preferential attachment model, where each new node attaches to m = 1 old nodes. Is such a network always connected?

In [34]:
```
n_small = 1050
m_small = 1
graph_small = ig.Graph.Barabasi(n=n_small, m=m_small, directed=False)
```

In [35]:
```
print(f"Is the network with n={n_small}, m={m_small} connected? {graph_small.is_con
```

```
Is the network with n=1050, m=1 connected? True
```

Yes, a preferential attachment network with n=1050 nodes and m=1 is always connected

Q1.2(b)

Use fast greedy method to find the community structure. Measure modularity. Define Assortativity. Compute Assortativity.

In [36]:
```
comm = graph_small.community_fastgreedy()
membership = comm.as_clustering()
print(f"Modularity: {membership.modularity:.4f}")
print(f"Assortativity: {graph_small.assortativity_degree(directed=False):.4f}")
```

```
Modularity: 0.9354
Assortativity: -0.0775
```

The modularity of this small graph is 0.9354.
Assortivity is the preference for the nodes of the network to connect with other nodes that are similar in some way. The assortivity is -0.0775 which means that high degree nodes tend to attach to lower degree nodes.

Q1.2(c)

Try to generate a larger network with 10500 nodes using the same model. Compute modularity and assortativity. How is it compared to the smaller network's modularity?

In [37]:
```
n_large = 10500
graph_large = ig.Graph.Barabasi(n=n_large, m=m_small, directed=False)

communities_large = graph_large.community_fastgreedy().as_clustering()
modularity_large = communities_large.modularity

print(f"Modularity: {modularity_large:.4f}")
print(f"Assortativity: {graph_large.assortativity_degree(directed=False):.4f}")
```

```
Modularity: 0.9785
Assortativity: -0.0372
```

Here we find that the modularity of the network is 0.9785 which is higher than the smaller network. Thus this network has a stronger community structure, with the nodes forming cohesive groups. The assortivity is -0.0372, which the closer to zero than the smaller graph, indicating that the larger network has a weaker disassortative mixing pattern, that is there is a lower tendency for the high-degree nodes to connect with the low-degree nodes

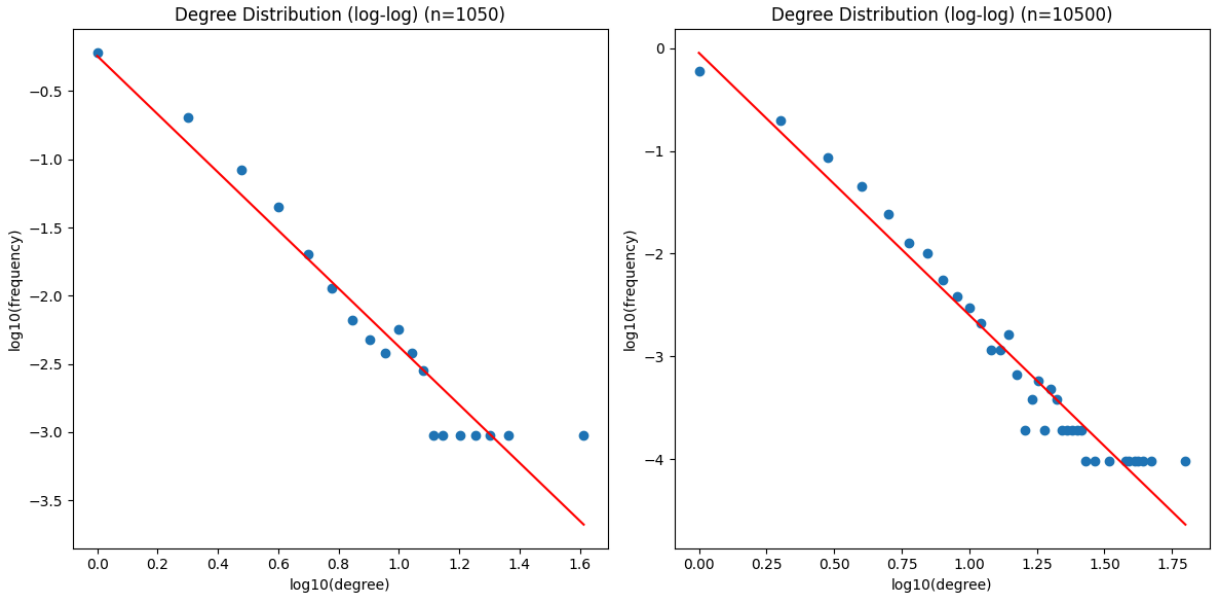
Q1.2(d)

Plot the degree distribution in a log-log scale for both n = 1050, 10500, then estimate the slope of the plot using linear regression.

In [ ]:
```
def plot_degree_distribution(graph, title):

    deg = graph.degree()
    counts = Counter(deg)
    k_vals = np.array(sorted(counts.keys()))
    freq = np.array([counts[k] / graph.vcount() for k in k_vals])
    mask = k_vals > 0
    log_k = np.log10(k_vals[mask])
    log_f = np.log10(freq[mask])

    slope, intercept = np.polyfit(log_k, log_f, 1)

    # Plot degree distribution
    plt.scatter(log_k, log_f, label="Data")
    plt.plot(log_k, slope * log_k + intercept, 'r-', label=f"Fit (slope={slope:.2f}

    plt.title(title)
    plt.xlabel("log10(degree)")
    plt.ylabel("log10(frequency)")
    # plt.title("Degree Distribution (log-log)")

    # Linear regression on log-log scale


    print(f"Slope of {title}: {slope:.4f}")
    return slope
```

In [61]:
```
# Plot degree distributions for both networks
plt.figure(figsize=(12, 6))

# Small network (n=1050)
plt.subplot(1, 2, 1)
slope_small = plot_degree_distribution(graph_small, f"Degree Distribution (log-log)

# Large network (n=10500)
plt.subplot(1, 2, 2)
slope_large = plot_degree_distribution(graph_large, f"Degree Distribution (log-log)

plt.tight_layout()
plt.show()

# Print slopes for comparison
print(f"Slope for n={n_small}: {slope_small:.4f}")
print(f"Slope for n={n_large}: {slope_large:.4f}")
```

```
Slope of Degree Distribution (log-log) (n=1050): -2.1298
Slope of Degree Distribution (log-log) (n=10500): -2.5537
```




```
Slope for n=1050: -2.1298
Slope for n=10500: -2.5537
```

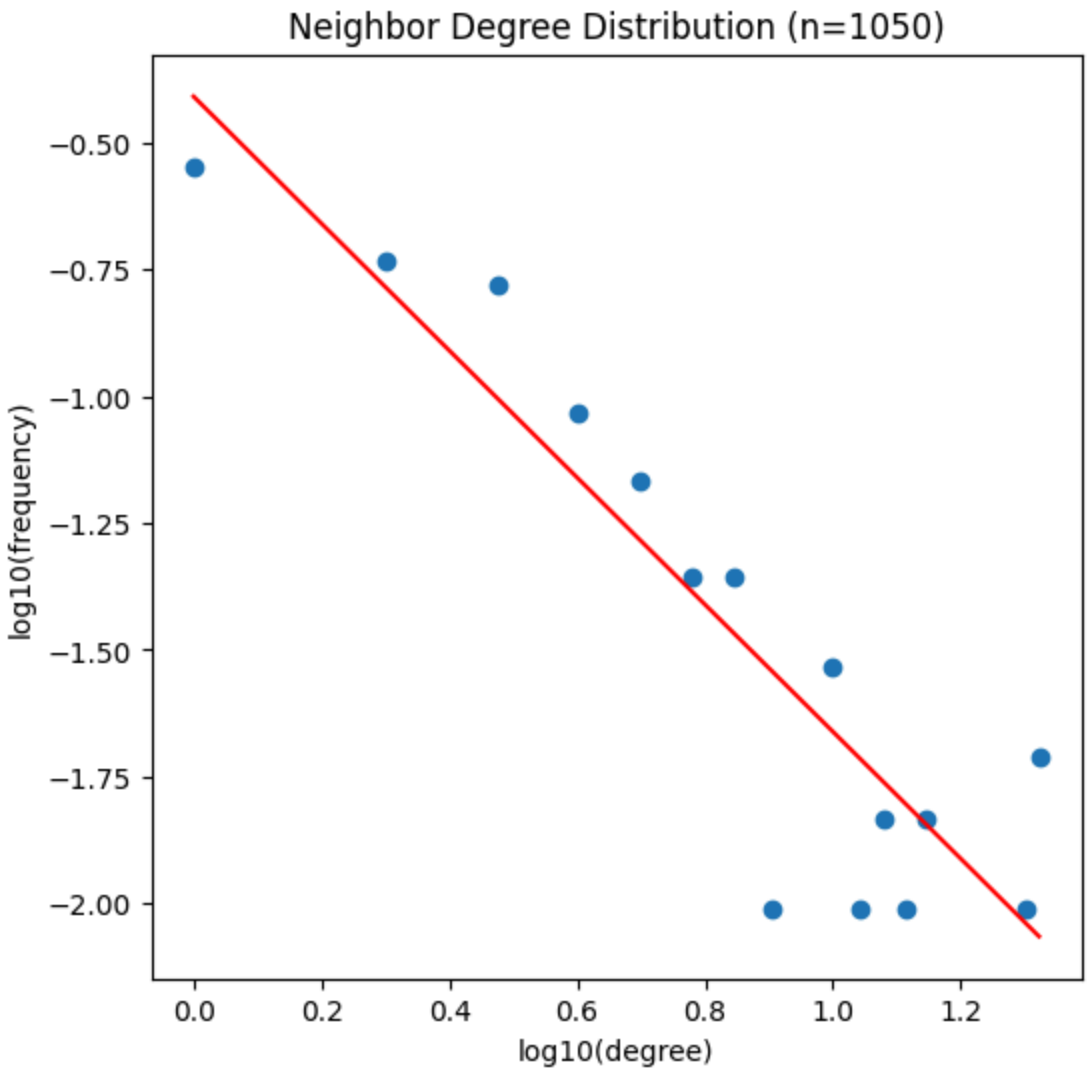
Q1.2(e)

In the two networks generated in 2(a) and 2(c), perform the following: Randomly pick a node i, and then randomly pick a neighbor j of that node. Plot the degree distribution of nodes j that are picked with this process, in the log-log scale. Is the distribution linear in the log-log scale? If so, what is the slope? How does this differ from the node degree distribution?

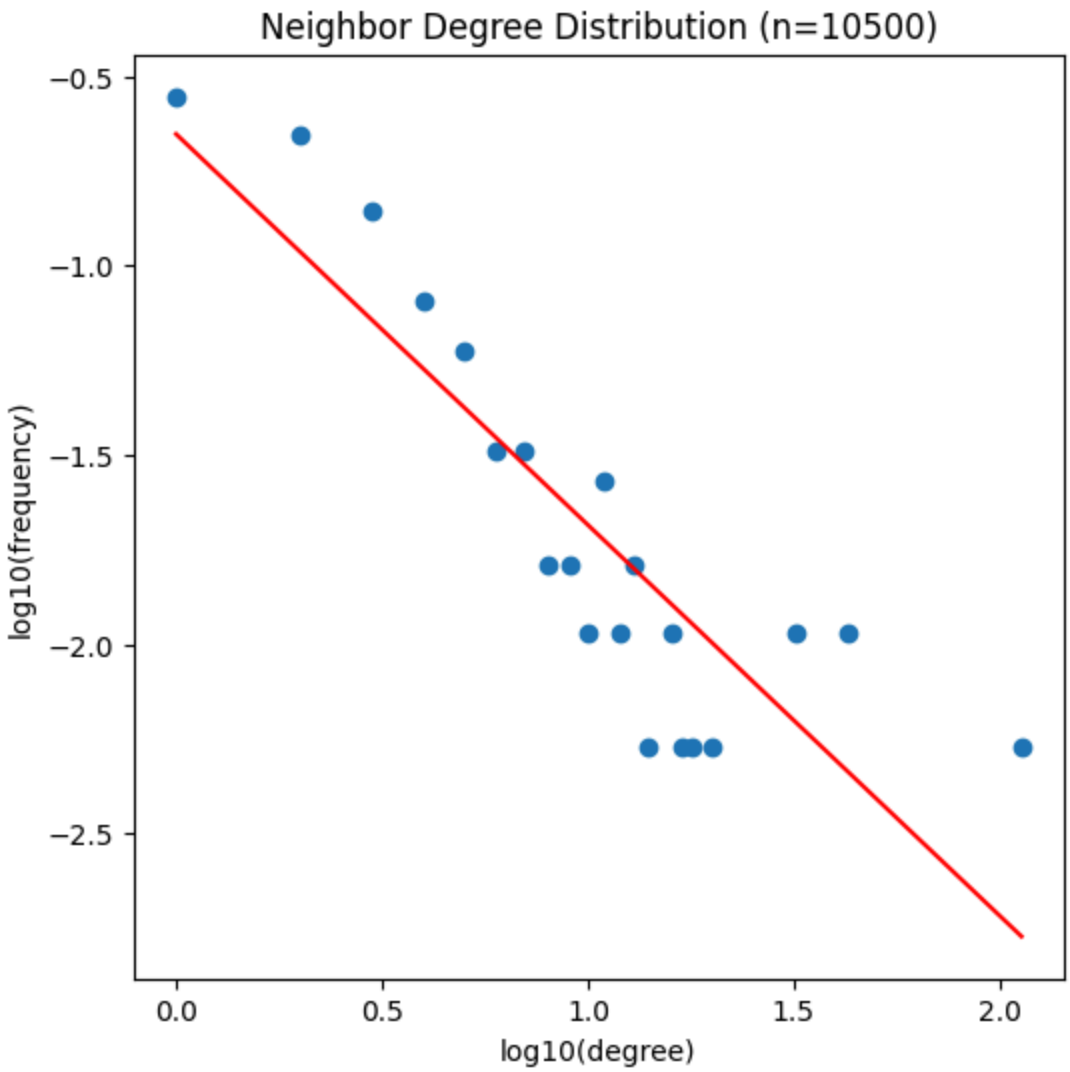
In [ ]:
```
def neighbor_degree_distribution(graph, graph_name, num_iterations=100):
    all_neighbor_degrees = []

    for _ in range(num_iterations):
        node_i = np.random.choice(graph.vcount())
        neighbors_j = graph.neighbors(node_i)

        if not neighbors_j:
            continue

        neighbor_degrees = [graph.degree(j) for j in neighbors_j]
        all_neighbor_degrees.extend(neighbor_degrees)

    if not all_neighbor_degrees:
        print(f"No neighbors found across all iterations for {graph_name}.")
        return None



    hist_neighbor_degrees = np.histogram(all_neighbor_degrees, bins=np.arange(1, ma
    x_neighbor = hist_neighbor_degrees[1][:-1]
    y_neighbor = hist_neighbor_degrees[0]

    x_neighbor = x_neighbor[y_neighbor > 0]
    y_neighbor = y_neighbor[y_neighbor > 0]
```

```
    log_k = np.log10(x_neighbor)
    log_f = np.log10(y_neighbor)

    slope_neighbor, intercept = np.polyfit(log_k, log_f, 1)

    if len(x_neighbor) < 2:
        print(f"Not enough data points for linear regression for {graph_name}.")
        return None

    plt.figure(figsize=(6, 6))  # Create a new figure for each graph
    # plt.loglog(x_neighbor, y_neighbor, 'o', markersize=4)
    plt.title(f"Neighbor Degree Distribution ({graph_name})")

    plt.scatter(log_k, log_f, label="Data")
    plt.plot(log_k, slope_neighbor * log_k + intercept, 'r-', label=f"Fit (slope={s
    plt.xlabel("log10(degree)")
    plt.ylabel("log10(frequency)")

    # slope_neighbor, intercept_neighbor, r_value_neighbor, p_value_neighbor, std_e
    print(f"Slope of neighbor degree distribution for {graph_name}: {slope_neighbor
    plt.show()  # Show the plot within the function
    return slope_neighbor

n_small = 1050
m_small = 1
graph_small = ig.Graph.Barabasi(n=n_small, m=m_small, directed=False)

n_large = 10500
graph_large = ig.Graph.Barabasi(n=n_large, m=m_small, directed=False)

neighbor_slope_small = neighbor_degree_distribution(graph_small, "n=1050", num_iter

neighbor_slope_large = neighbor_degree_distribution(graph_large, "n=10500", num_ite
```

```
Slope of neighbor degree distribution for n=1050: -1.2521
```




```
Slope of neighbor degree distribution for n=10500: -1.0319
```



The first thing we see is that the slope of the degree distribution is less negative when choosing a random neighbour, than the slope of the entire network.

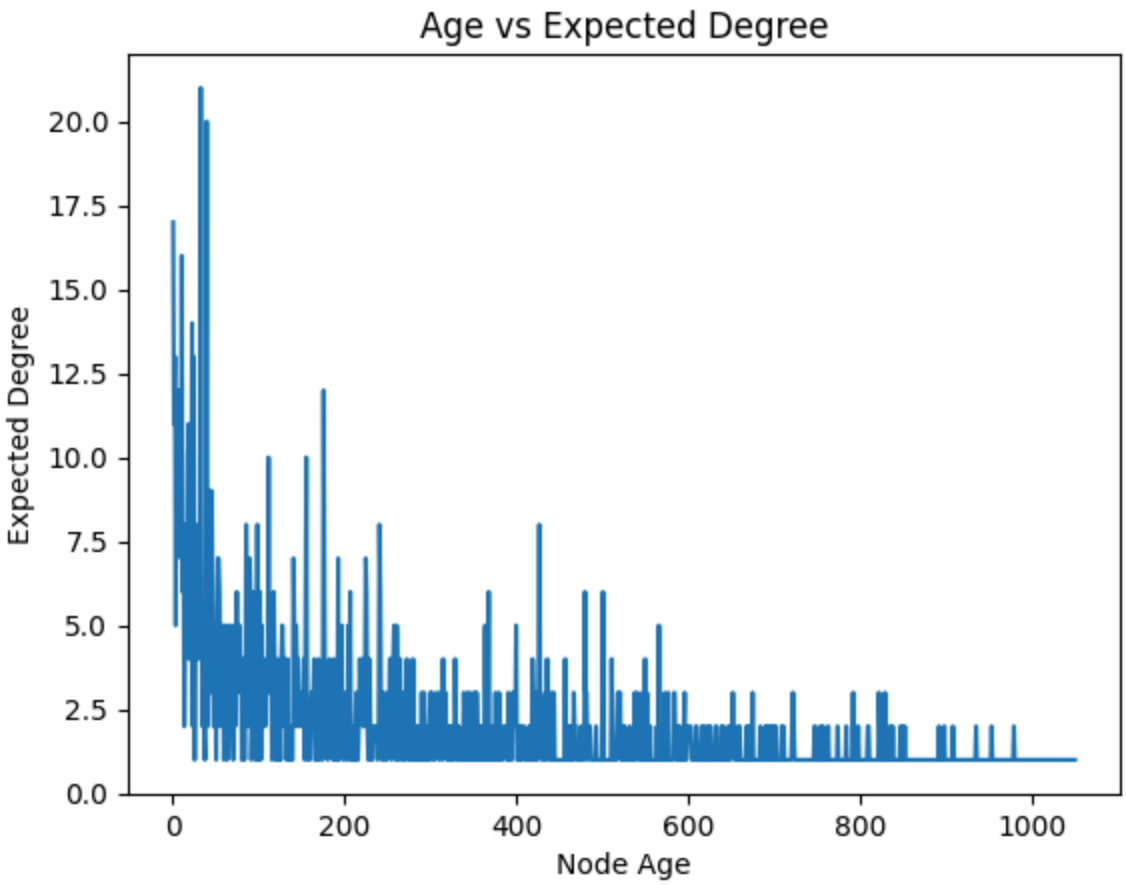
We also see that the slope of the larger network has a more negative slope than the smaller network. This indicates that there is more disassortative mixing than the neighbours of the smaller network

Q1.2 (f)

Estimate the expected degree of a node that is added at time step i for 1 ≤ i ≤ 1050. Show the relationship between the age of nodes and their expected degree through an appropriate plot. Note that the newest added node is the youngest.

In [69]:
```
ages = np.arange(1, n_small + 1)
expected_degrees = [graph_small.degree(i) for i in range(n_small)]
plt.plot(ages, expected_degrees)
plt.title("Age vs Expected Degree")
plt.xlabel("Node Age")
```

```
plt.ylabel("Expected Degree")
plt.show()
```



Older nodes that higher degrees due to preferential attachement, while the younger nodes have lower degrees. These means that the older nodes dominate the hubs, being connected to more nodes over time

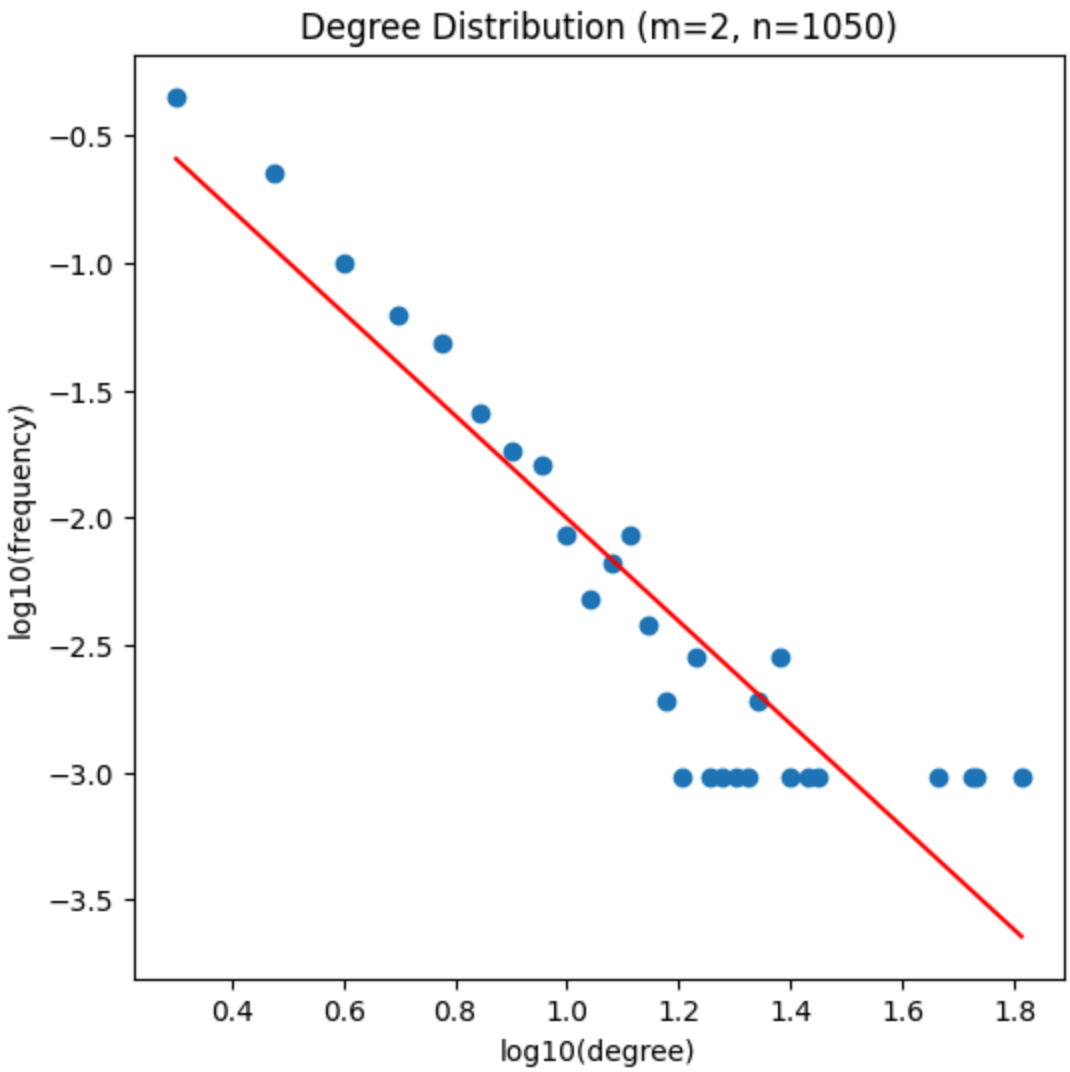
Q1.2 (g)

Repeat the previous parts (a-f) for m = 2, and m = 6. Compare the results of each part for different values of m.

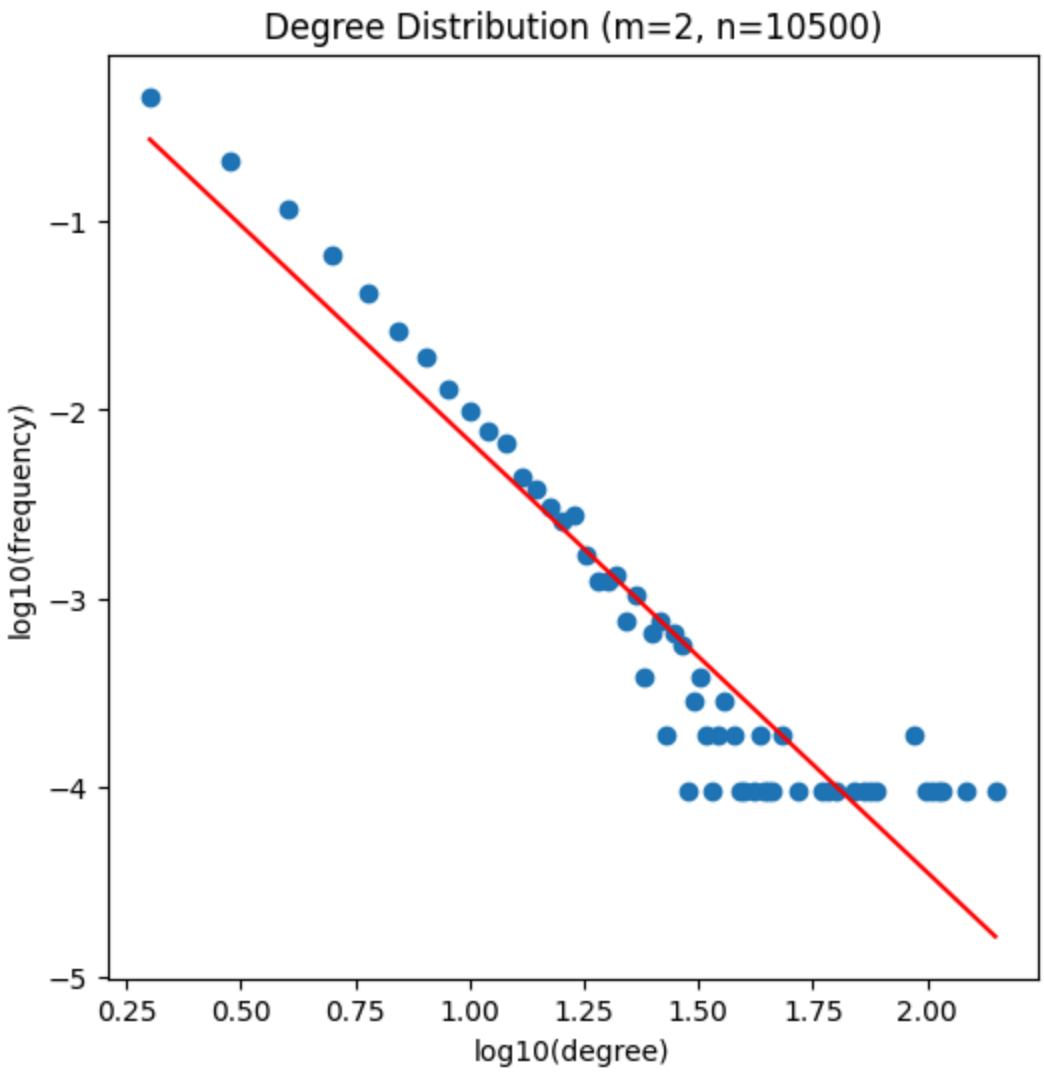
In [70]:
```
for m in [2, 6]:
    print(f"\n--- Analyzing for m = {m} ---")

    # a: Create graph (n=1050)
    n_small = 1050
    graph_small = ig.Graph.Barabasi(n=n_small, m=m, directed=False)

    # c: Create graph (n=10500)
    n_large = 10500
    graph_large = ig.Graph.Barabasi(n=n_large, m=m, directed=False)

    # b: Community structure (n=1050)
    communities_m = graph_small.community_fastgreedy().as_clustering()
    modularity_m = communities_m.modularity
```

```
    assortativity_m = graph_small.assortativity_degree(directed=False)

    print(f"Modularity (n=1050): {modularity_m:.4f}")
    print(f"Assortativity (n=1050): {assortativity_m:.4f}")

    # Community structure (n=10500)
    communities_m_large = graph_large.community_fastgreedy().as_clustering()
    modularity_m_large = communities_m_large.modularity
    assortativity_m_large = graph_large.assortativity_degree(directed=False)

    print(f"Modularity (n=10500): {modularity_m_large:.4f}")
    print(f"Assortativity (n=10500): {assortativity_m_large:.4f}")

    # d: Degree distribution (n=1050)
    plt.figure(figsize=(6, 6))
    slope_m = plot_degree_distribution(graph_small, f"Degree Distribution (m={m}, n
    plt.show()

    # d: Degree distribution (n=10500)
    plt.figure(figsize=(6, 6))
    slope_m_large = plot_degree_distribution(graph_large, f"Degree Distribution (m=
    plt.show()

    # e: Neighbor degree distribution (n=1050)
    neighbor_slope_small = neighbor_degree_distribution(graph_small, f"m={m}, n=105

   # e: Neighbor degree distribution (n=10500)
    neighbor_slope_large = neighbor_degree_distribution(graph_large, f"m={m}, n=105

    # f: Age vs expected degree (n=1050)
    ages = np.arange(1, n_small + 1)
    expected_degrees = [graph_small.degree(i) for i in range(n_small)]
    plt.figure(figsize=(6, 6))
    plt.plot(ages, expected_degrees)
    plt.title(f"Age vs Expected Degree (m={m}, n=1050)")
    plt.xlabel("Node Age")
    plt.ylabel("Expected Degree")
    plt.show()
```

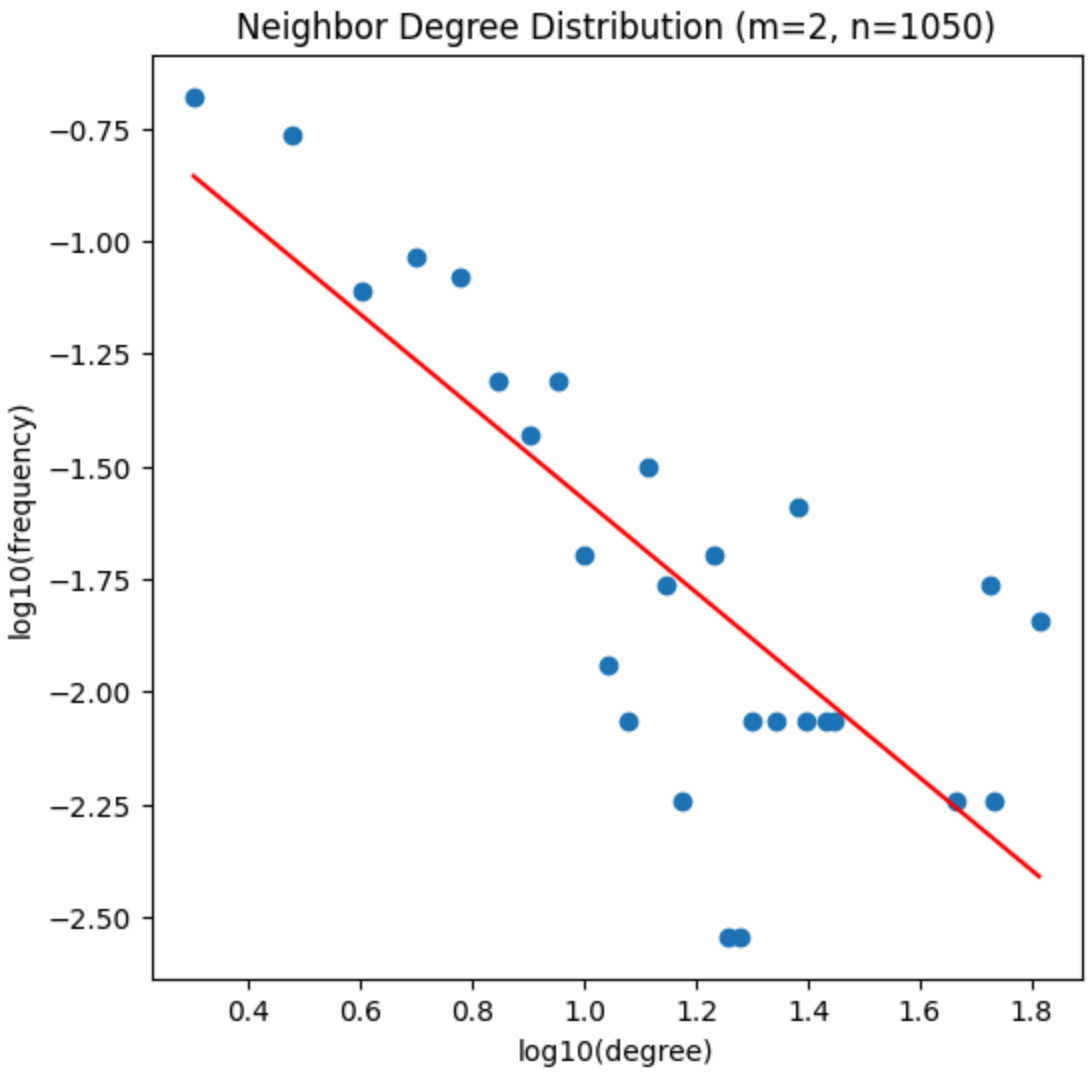
```
--- Analyzing for m = 2 ---
Modularity (n=1050): 0.5243
Assortativity (n=1050): -0.0324
Modularity (n=10500): 0.5305
Assortativity (n=10500): -0.0263
Slope of Degree Distribution (m=2, n=1050): -2.0192
```



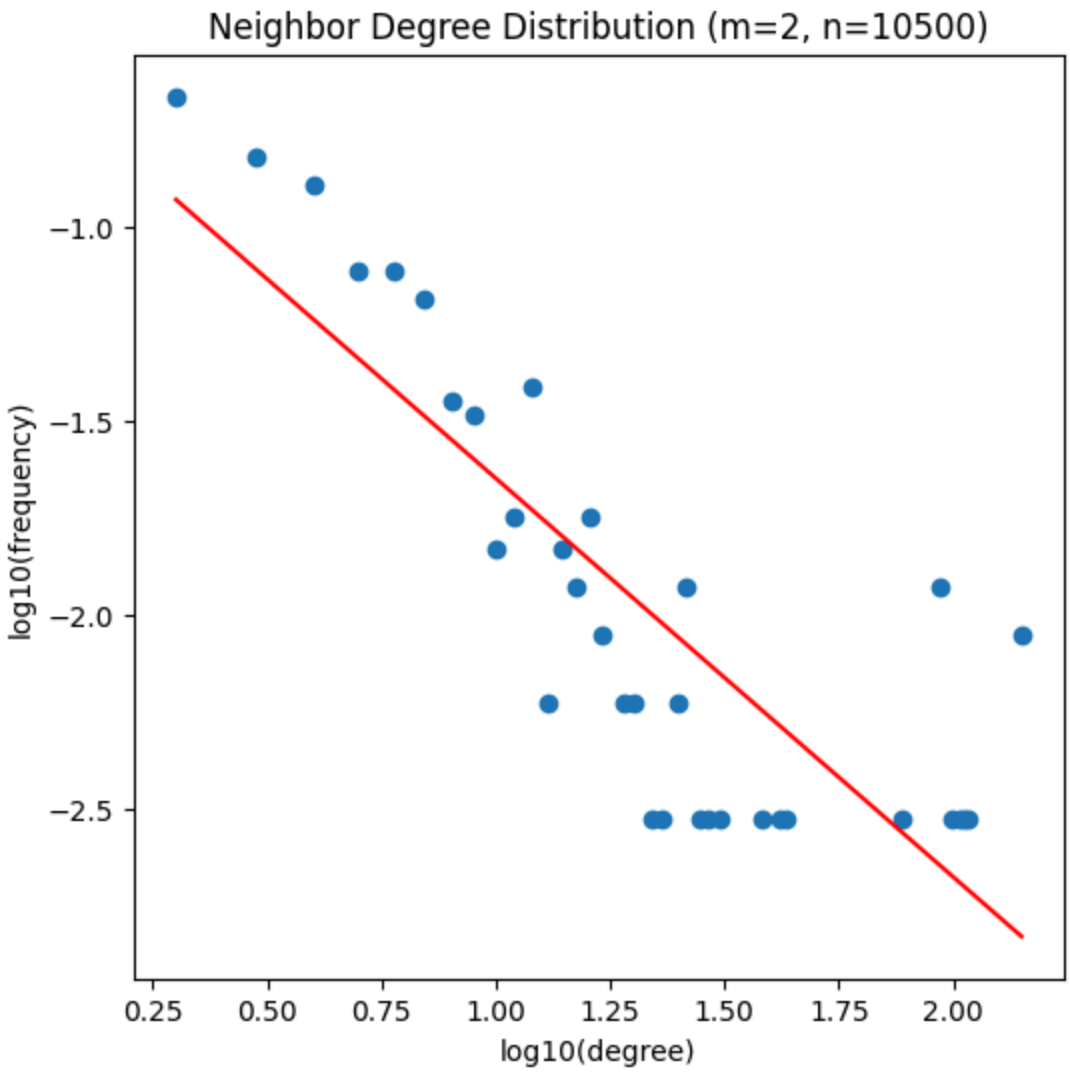

```
Slope of Degree Distribution (m=2, n=10500): -2.2833
```

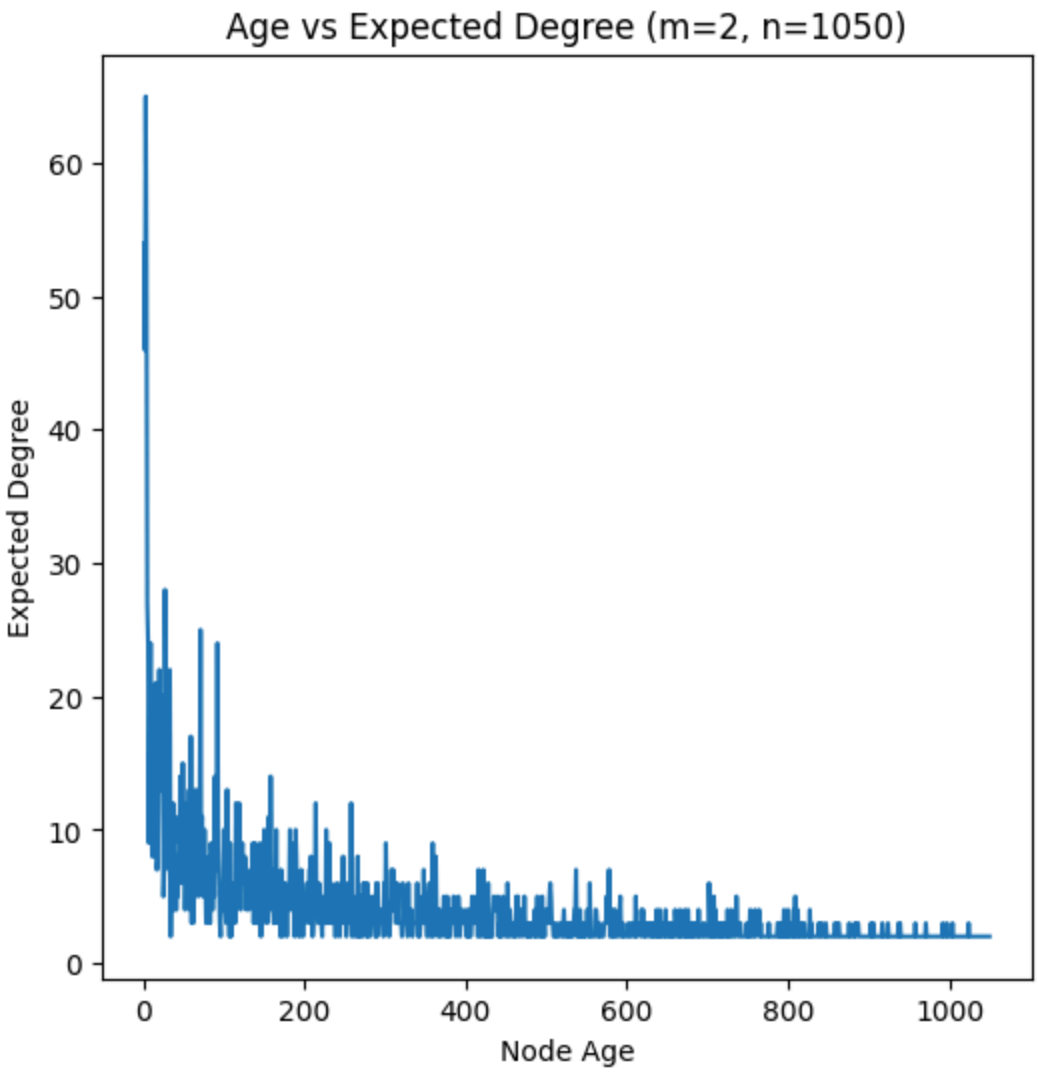



```
Slope of neighbor degree distribution for m=2, n=1050: -1.0276
```

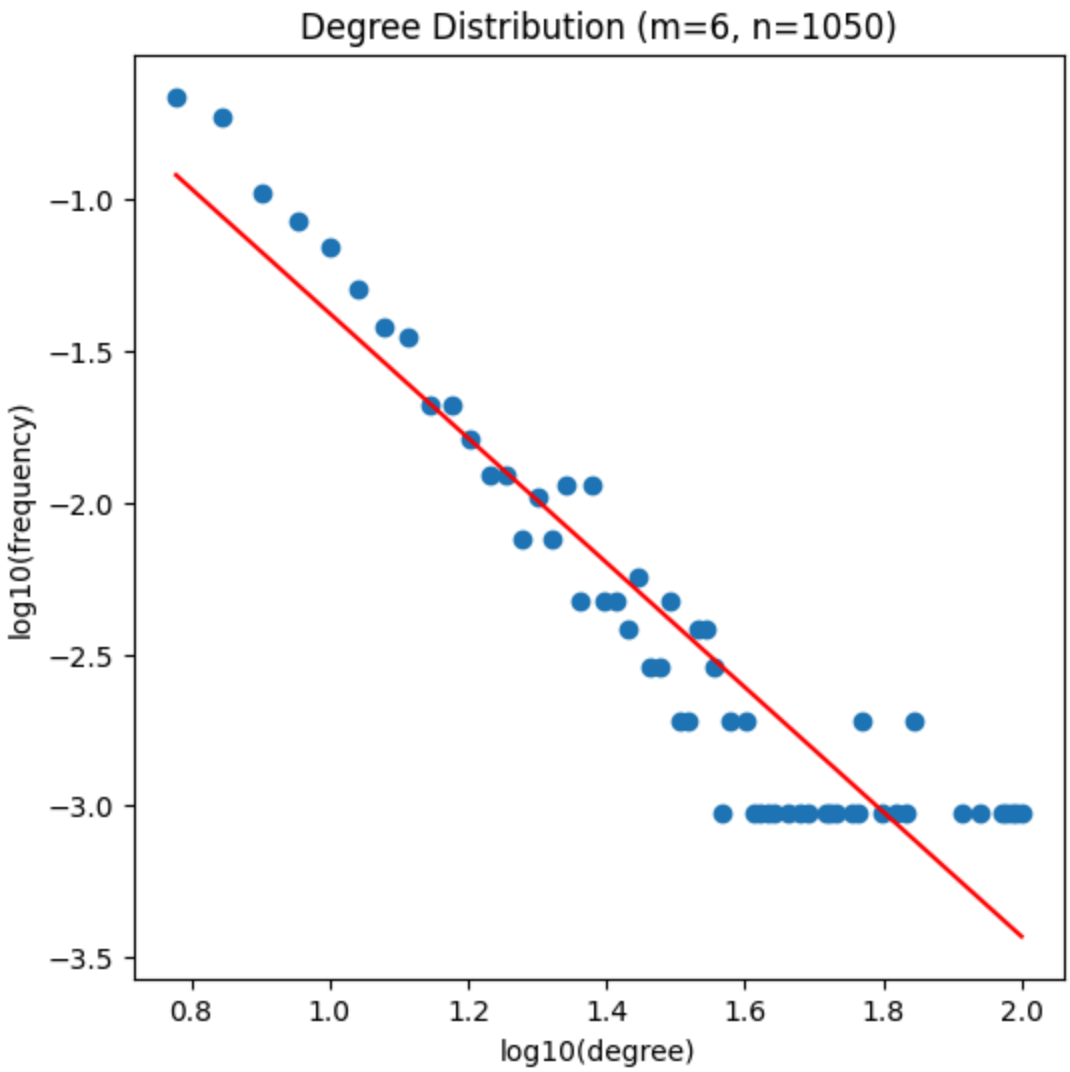



```
Slope of neighbor degree distribution for m=2, n=10500: -1.0269
```

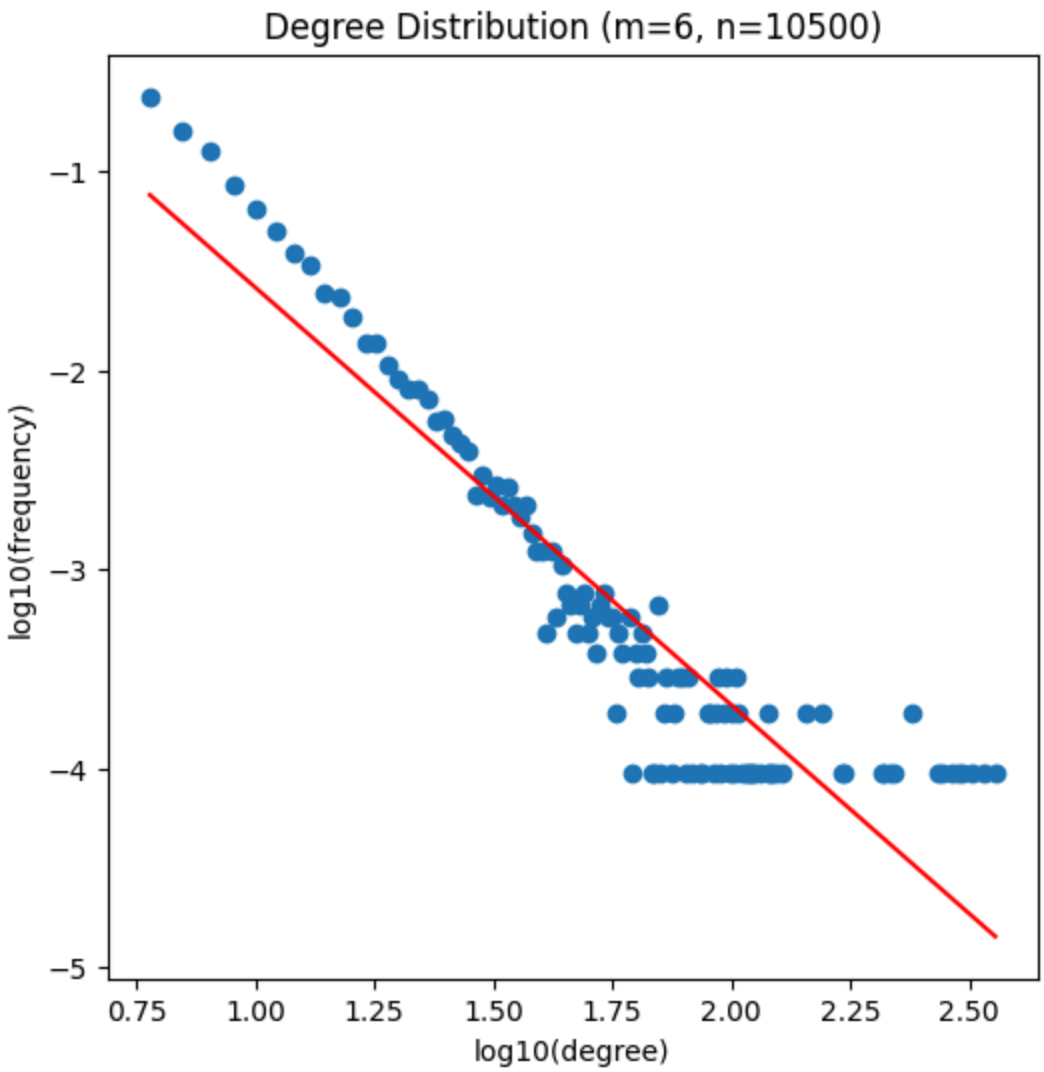

Neighbor Degree Distribution (m=2, n=10500)
log10(frequency)
−1.0
−1.5
−2.0
−2.5
0.25
0.50
0.75
1.00
1.25
1.50
1.75
2.00
log10(degree)



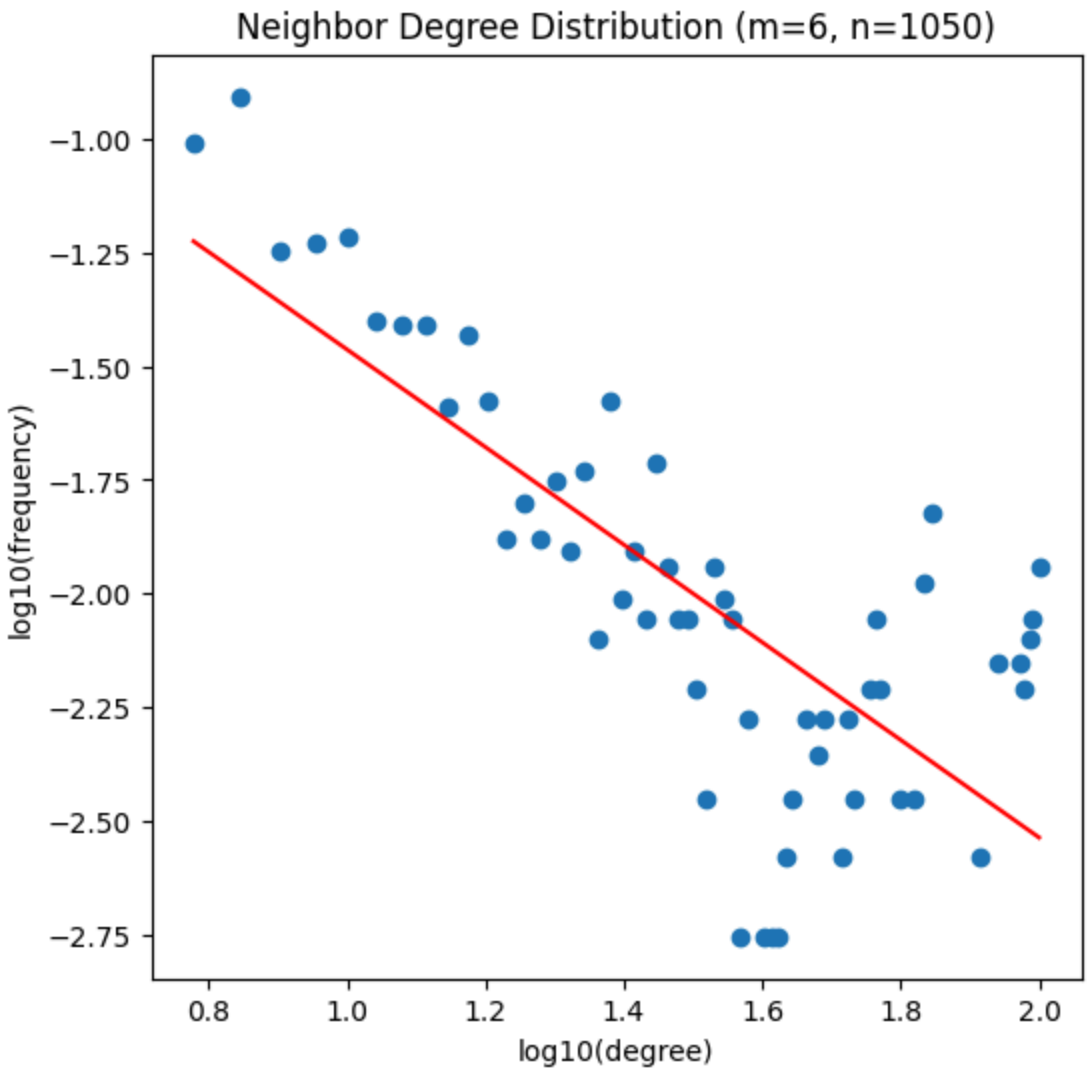

```
--- Analyzing for m = 6 ---
Modularity (n=1050): 0.2423
Assortativity (n=1050): -0.0084
Modularity (n=10500): 0.2505
Assortativity (n=10500): -0.0050
Slope of Degree Distribution (m=6, n=1050): -2.0563
```



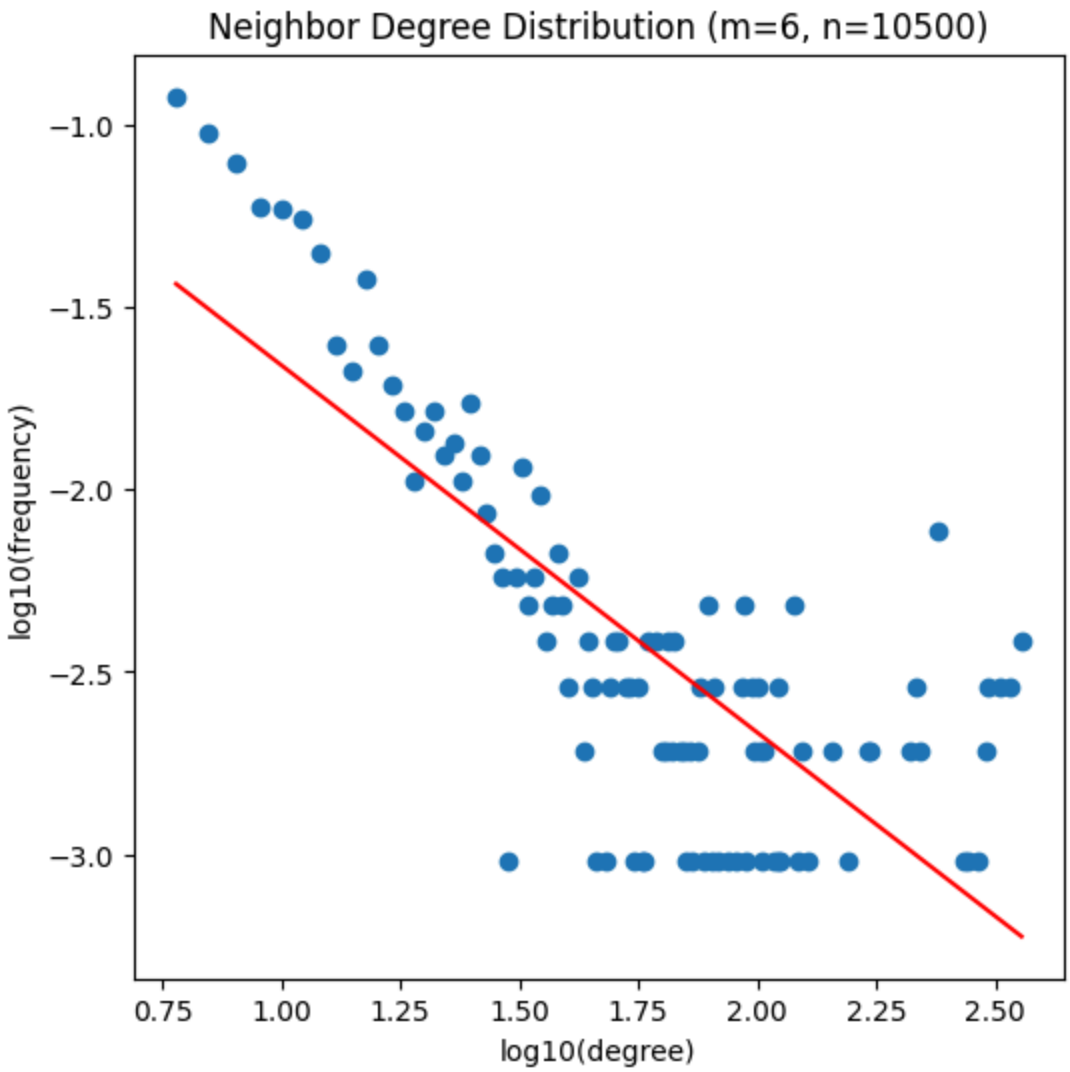

```
Slope of Degree Distribution (m=6, n=10500): -2.0946
```

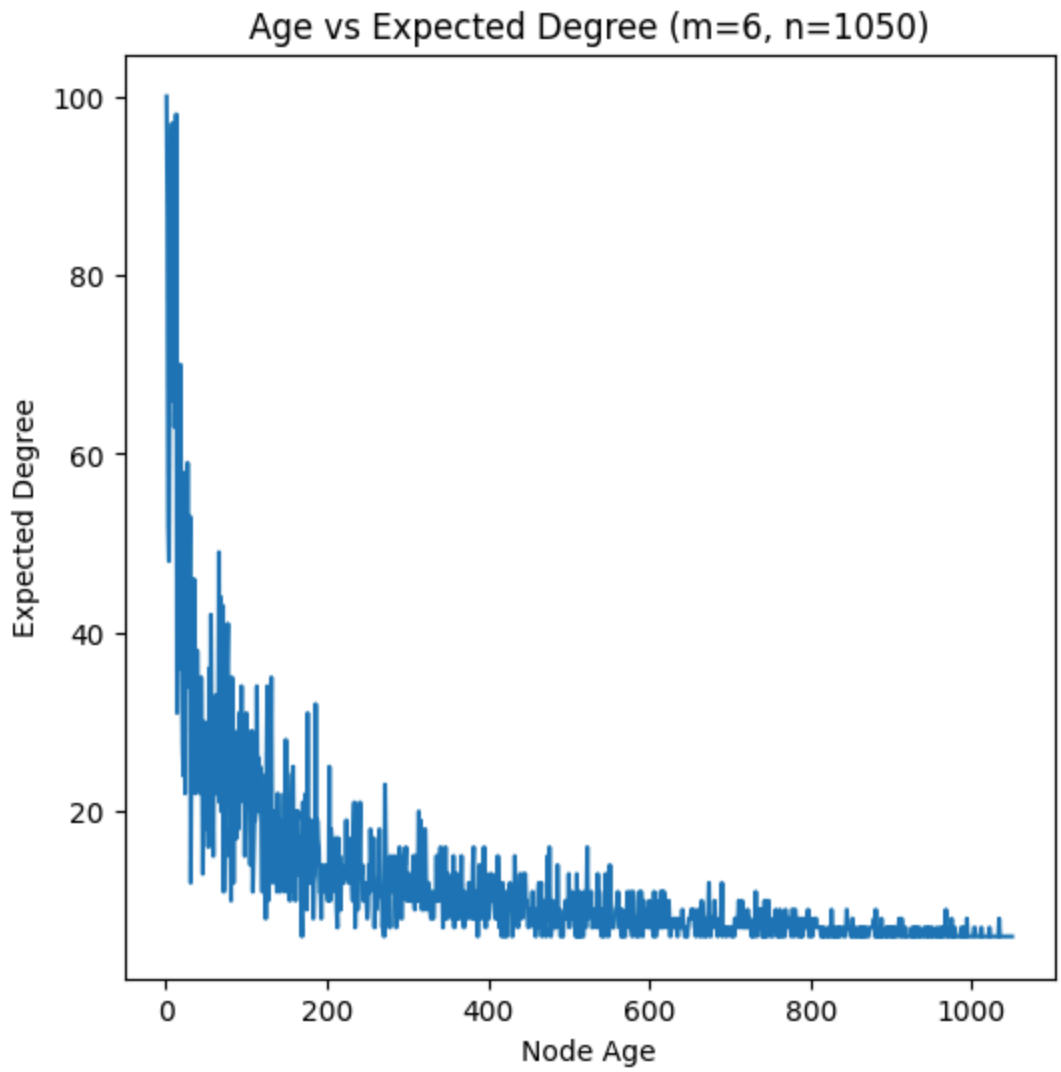



```
Slope of neighbor degree distribution for m=6, n=1050: -1.0718
```




```
Slope of neighbor degree distribution for m=6, n=10500: -1.0055
```


Neighbor Degree Distribution (m=6, n=10500)
−1.0
−1.5
−2.0
−2.5
−3.0
log10(frequency)
0.75
1.00
1.25
1.50
1.75
2.00
2.25
2.50
log10(degree)



Comparing between m=2 and m=6

Q1.2(h)

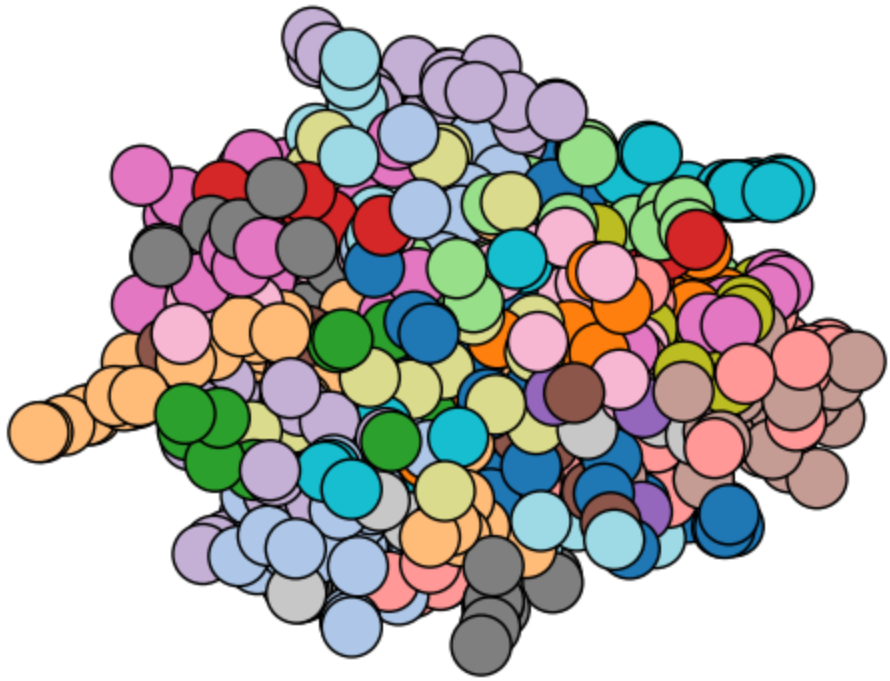
Again, generate a preferential attachment network with n = 1050, m = 1. Take its degree sequence and create a new network with the same degree sequence, through stub-matching procedure. Plot both networks, mark communities on their plots, and measure their modularity. Compare the two procedures for creating random power-law networks.

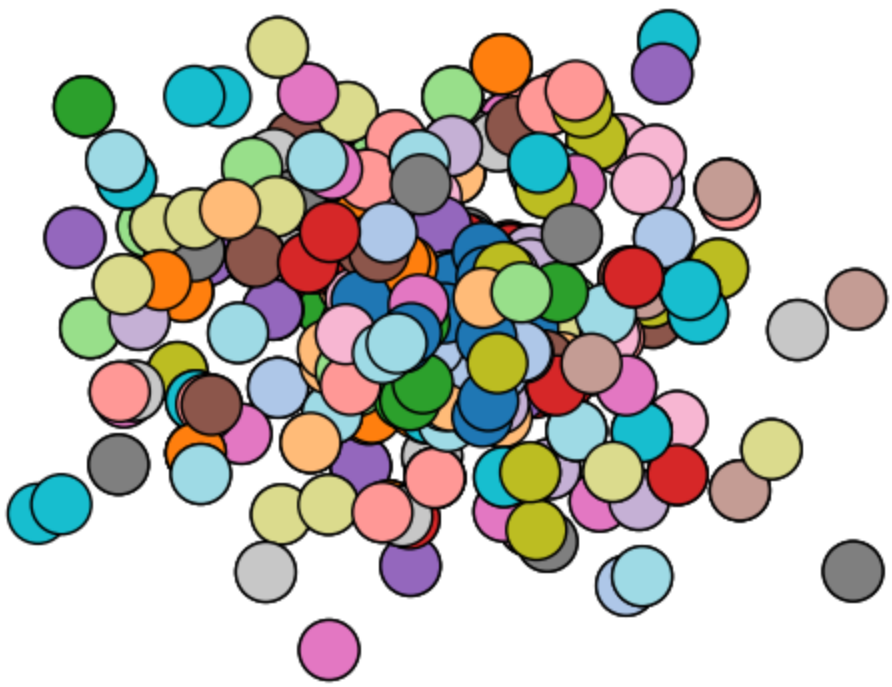
In [76]:
```
import igraph as ig
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import cm
from matplotlib.colors import to_hex

# (h) Stub-matching procedure
n_small = 1050
m_small = 1
graph_small = ig.Graph.Barabasi(n=n_small, m=m_small, directed=False)
```

```python
communities_small = graph_small.community_fastgreedy().as_clustering()
modularity_small = communities_small.modularity

degree_sequence = graph_small.degree()
stub_matched_graph = ig.Graph.Degree_Sequence(degree_sequence, method="simple")

communities_stub_matched = stub_matched_graph.community_walktrap().as_clustering()
modularity_stub_matched = communities_stub_matched.modularity

print(f"\nStub-matched network modularity: {modularity_stub_matched:.4f}")
print(f"Original network modularity: {modularity_small:.4f}")

# Plot original and stub-matched networks with communities marked
layout_original = graph_small.layout_fruchterman_reingold()
layout_stub_matched = stub_matched_graph.layout_fruchterman_reingold()

def membership_to_colors(membership):
    n_comms = max(membership) + 1
    cmap = cm.get_cmap("tab20", n_comms)   # discrete colormap
    return [to_hex(cmap(c)) for c in membership]

colors_original = membership_to_colors(communities_small.membership)
colors_stub = membership_to_colors(communities_stub_matched.membership)
# Plot original network
fig, ax = plt.subplots()
ig.plot(graph_small, target=ax, layout=layout_original,
        vertex_color=colors_original,
        title="Original Network with Communities")
plt.show()

# Plot stub-matched network
fig, ax = plt.subplots()
ig.plot(stub_matched_graph, target=ax, layout=layout_stub_matched,
        vertex_color=colors_stub,
        title="Stub-Matched Network with Communities")
plt.show()
```

```
Stub-matched network modularity: 0.7154
Original network modularity: 0.9338
```

```
C:\Users\Anwesha\AppData\Local\Temp\ipykernel_71488\2329496433.py:30: MatplotlibDepr
ecationWarning: The get_cmap function was deprecated in Matplotlib 3.7 and will be r
emoved in 3.11. Use ``matplotlib.colormaps[name]`` or ``matplotlib.colormaps.get_cma
p()`` or ``pyplot.get_cmap()`` instead.
  cmap = cm.get_cmap("tab20", n_comms)   # discrete colormap
```





Q1.3 Create a modified preferential attachment model that penalizes the age of a node

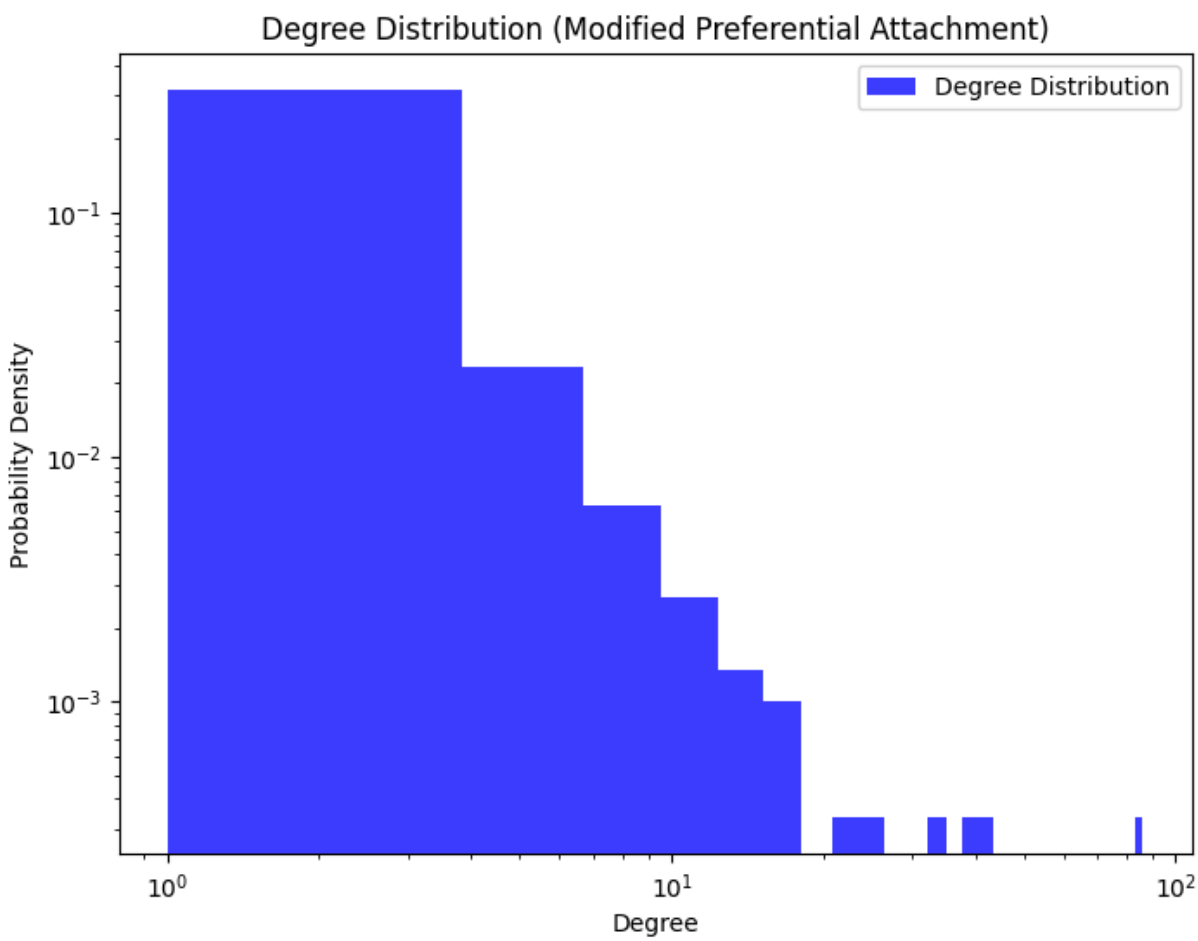
1.3 (a)

(a) Each time a new vertex is added, it creates m links to old vertices and the probability that an old vertex is cited depends on its degree (preferential attachment) and age. In particular, the probability that a newly added vertex connects to an old vertex is proportional to:

where ki is the degree of vertex i in the current time step, and li is the age of vertex i. Produce such an undirected network with 1050 nodes and parameters m = 1, α = 1, β = −1, and a = c = d = 1, b = 0. Plot the degree distribution. What is the power law exponent?

In [78]:
```
import powerlaw

N = 1050  # Total nodes
m = 1     # Each new node connects to 1 existing node
alpha = 1
beta = -1
a = c = d = 1
b = 0

# Initialize network with 2 connected nodes
G = nx.Graph()
G.add_edges_from([(0, 1)])

# Track node ages (initialize all nodes with age = 1)
ages = {0: 1, 1: 1}

# Preferential attachment function with age penalty
def attachment_prob(G, ages):
    degrees = np.array([G.degree(n) for n in G.nodes()])

    # Ensure every node has an age entry
    for node in G.nodes():
        if node not in ages:
            ages[node] = 1  # Initialize new nodes with age 1

    ages_inv = np.array([1 / ages[n] for n in G.nodes()])  # Age factor l_i^β
    probs = (c * degrees**alpha + a) * (d * ages_inv**beta + b)
    return probs / np.sum(probs)  # Normalize

# Grow the network
for new_node in range(2, N):
    G.add_node(new_node)

    # Ensure new node is added to ages dictionary
    ages[new_node] = 1  # New node starts at age 1

    probs = attachment_prob(G, ages)

    # Choose an existing node based on probability
    old_nodes = list(G.nodes())
    selected_node = np.random.choice(old_nodes, p=probs)

    # Connect the new node
    G.add_edge(new_node, selected_node)

    # Update ages of all existing nodes
```

```
    for node in G.nodes():
        ages[node] += 1

# Extract degrees
degrees = [G.degree(n) for n in G.nodes()]

# Plot Degree Distribution
plt.figure(figsize=(8,6))
plt.hist(degrees, bins=30, density=True, alpha=0.75, color='b', label="Degree Distr
plt.xlabel("Degree")
plt.ylabel("Probability Density")
plt.xscale("log")
plt.yscale("log")
plt.title("Degree Distribution (Modified Preferential Attachment)")
plt.legend()
plt.show()

# Fit Power Law
fit = powerlaw.Fit(degrees, discrete=True)
power_law_exponent = fit.power_law.alpha
print(f"Power Law Exponent: {power_law_exponent:.2f}")
```




```
Calculating best minimal value for power law fit
Fitting xmin: 100%|██████████| 19/19 [00:00<00:00, 724.24it/s]
Power Law Exponent: 2.35
```

```
d:\ucla_coursework\CURRENT\232E\232E_sp26\.venv\Lib\site-packages\powerlaw\distribut
ions.py:743: OptimizeWarning: Initial guess is not within the specified bounds
  result = scipy.optimize.minimize(fit_function,
```

The degree distribution follows a power law decay. The alpha value is 2.35. Adding the age penalty prevents the older nodes from dominating the network.

Q1.3 (b)

Use fast greedy method to find the community structure. What is the modularity?

In [81]:
```
igraph_G = ig.Graph.TupleList(G.edges(), directed=False)
community = igraph_G.community_fastgreedy()
modularity = community.as_clustering().modularity
print(f"Modularity of the network: {modularity:.4f}")
```

```
Modularity of the network: 0.9186
```

The value of a the modularity of the network suggests that the network is highly structured, with distinct groups over time

Q2 Random Walk on networks

Q 2.1 Random walk on Erd¨os-R´enyi networks

- (a) Create an undirected random network with 900 nodes, and the probability p for drawing an edge between any pair of nodes equal to 0.015.
- (b)Let a random walker start from a randomly selected node (no teleportation). We use t to denote the number of steps that the walker has taken. Measure the average distance (defined as the shortest path length) ⟨s(t)⟩ of the walker from his starting point at step t. Also, measure the variance σ2(t) = ⟨(s(t)−⟨s(t)⟩)2⟩ of this distance. Plot ⟨s(t)⟩ v.s. t and σ2(t) v.s. t. Here, the average ⟨·⟩ is over random choices of the starting nodes.
- (c) Measure the degree distribution of the nodes reached at the end of the random walk. How does it compare to the degree distribution of graph?
- (d) Repeat 1(b) for undirected random networks with 9000 nodes. Compare the results and explain qualitatively. Does the diameter of the network play a role?

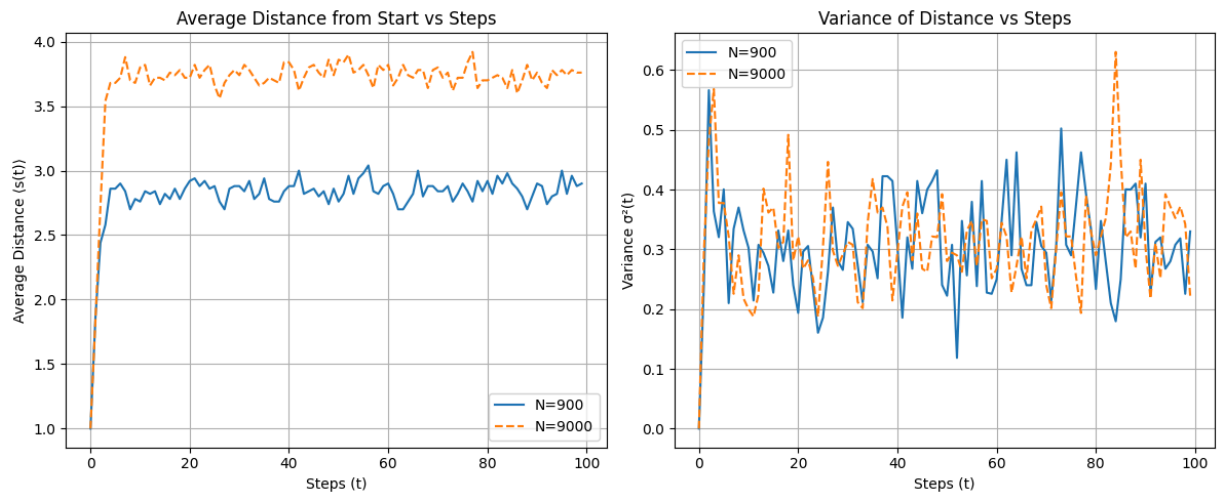
In [ ]:
```
import igraph as ig
import numpy as np
import matplotlib.pyplot as plt
import random

# Parameters
N1, p1 = 900, 0.015     # Small ER network
N2, p2 = 9000, 0.0015   # Large ER network (same expected degree)
steps = 100             # Number of steps in the random walk
trials = 50             # Number of random walk trials
```

```python
def create_er_network(N, p):
    return ig.Graph.Erdos_Renyi(n=N, p=p)

def random_walk(G, steps, trials):
    """
    Perform random walks starting from random nodes in the graph G.
    Measure:
    - Average distance ⟨s(t)⟩ from start node
    - Variance σ²(t) of distance
    - Final nodes visited (to analyze degree distribution)
    """
    avg_distance = np.zeros(steps)
    var_distance = np.zeros(steps)
    final_nodes = []

    nodes = list(range(G.vcount()))

    for _ in range(trials):
        start_node = random.choice(nodes)
        current_node = start_node
        distances = []

        for t in range(steps):
            neighbors = G.neighbors(current_node)
            if not neighbors:
                # Stay in place if isolated
                distances.append(distances[-1] if distances else 0)
                continue

            next_node = random.choice(neighbors)
            current_node = next_node

            # Shortest path distance from start
            dist = G.shortest_paths(source=start_node, target=current_node)[0][0]
            distances.append(dist)

        if len(distances) < steps:
            distances.extend([distances[-1]] * (steps - len(distances)))

        distances = np.array(distances)
        avg_distance += distances
        var_distance += distances ** 2
        final_nodes.append(current_node)

    avg_distance /= trials
    var_distance = var_distance / trials - avg_distance ** 2
    return avg_distance, var_distance, final_nodes

def degree_distribution(G, sample_nodes=None, title="Degree Distribution"):
    """
    Plot degree distribution:
    - Full graph if sample_nodes is None
    - Else plot distribution of provided node list
    """
    if sample_nodes is not None:
```

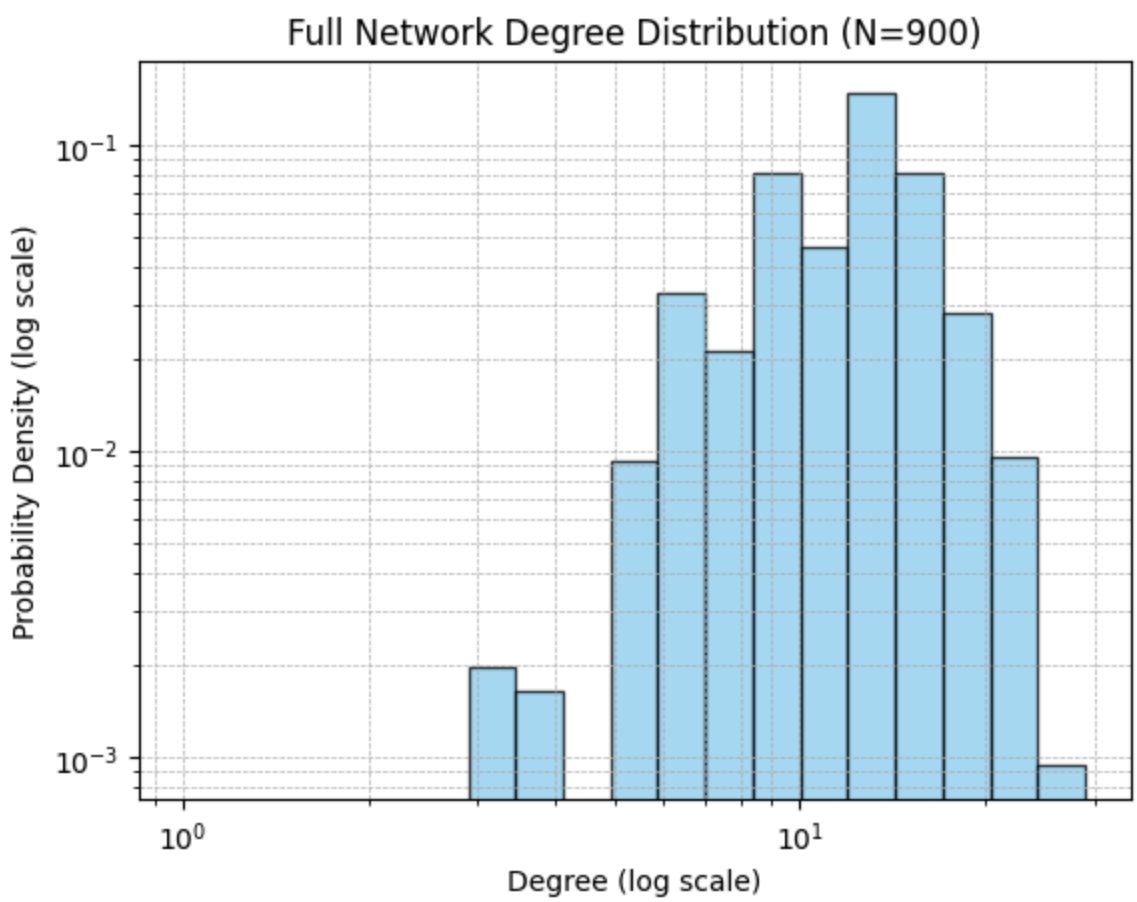
```
        degrees = [G.degree(n) for n in sample_nodes]
    else:
        degrees = G.degree()

    max_deg = max(degrees) if len(degrees) > 0 else 1

    plt.figure()
    plt.hist(
        degrees,
        bins=np.logspace(np.log10(1), np.log10(max_deg + 1), 20),
        density=True,
        color='skyblue',
        edgecolor='black',
        alpha=0.75
    )
    plt.xscale('log')
    plt.yscale('log')
    plt.xlabel('Degree (log scale)')
    plt.ylabel('Probability Density (log scale)')
    plt.title(title)
    plt.grid(True, which="both", ls="--", linewidth=0.5)
    plt.show()

# (a) Create ER graphs
G1 = create_er_network(N1, p1)
G2 = create_er_network(N2, p2)

# (b) Random walk analysis for N=900
s_avg_900, s_var_900, visited_nodes_900 = random_walk(G1, steps, trials)

# (d) Repeat for N=9000
s_avg_9000, s_var_9000, visited_nodes_9000 = random_walk(G2, steps, trials)

# Plotting ⟨s(t)⟩ and σ²(t)
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
plt.plot(range(steps), s_avg_900, label='N=900')
plt.plot(range(steps), s_avg_9000, label='N=9000', linestyle='dashed')
plt.xlabel('Steps (t)')
plt.ylabel('Average Distance ⟨s(t)⟩')
plt.title('Average Distance from Start vs Steps')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(range(steps), s_var_900, label='N=900')
plt.plot(range(steps), s_var_9000, label='N=9000', linestyle='dashed')
plt.xlabel('Steps (t)')
plt.ylabel('Variance σ²(t)')
plt.title('Variance of Distance vs Steps')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

```
# (c) Degree distribution comparison
print("Degree distribution of full network (N=900):")
degree_distribution(G1, title="Full Network Degree Distribution (N=900)")

print("Degree distribution of visited nodes after random walk (N=900):")
degree_distribution(G1, sample_nodes=visited_nodes_900, title="Visited Nodes Degree
```

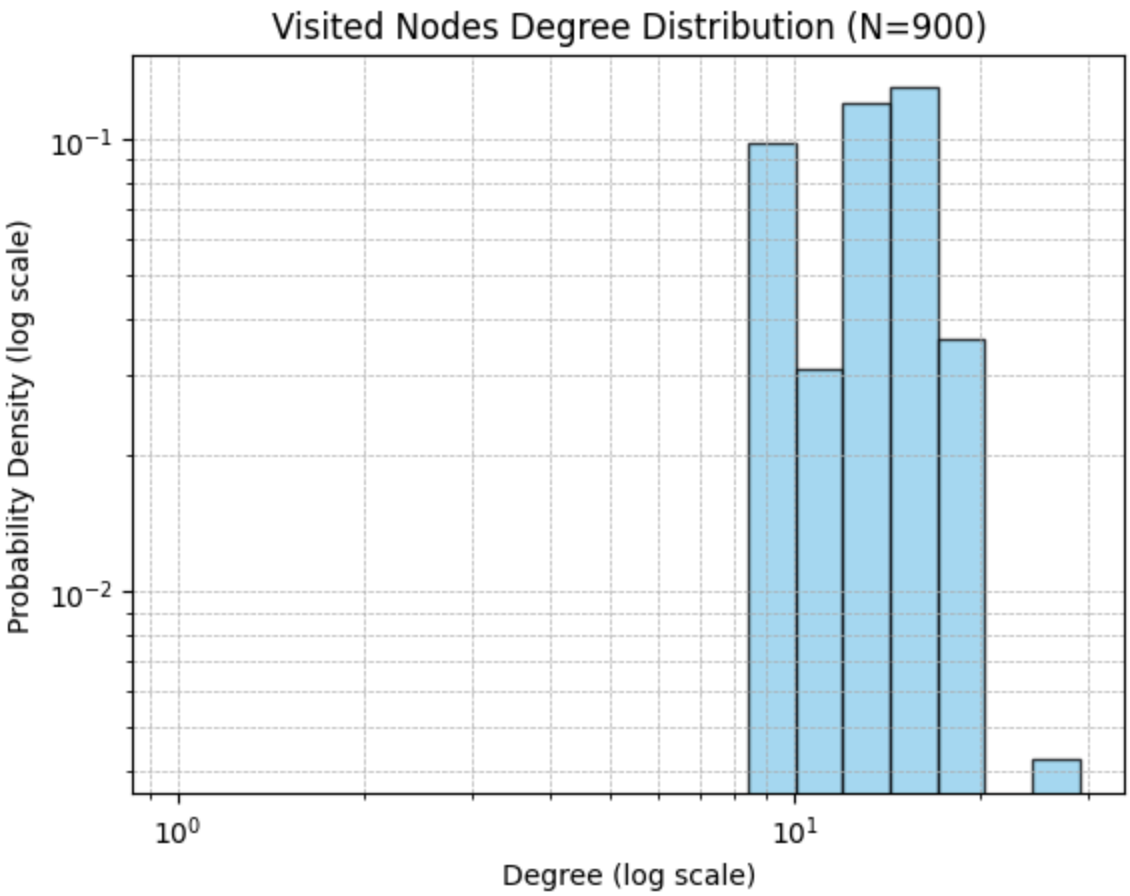
```
C:\Users\Anwesha\AppData\Local\Temp\ipykernel_71488\1810743539.py:46: DeprecationWar
ning: Graph.shortest_paths() is deprecated; use Graph.distances() instead
  dist = G.shortest_paths(source=start_node, target=current_node)[0][0]
```




```
Degree distribution of full network (N=900):
```




```
Degree distribution of visited nodes after random walk (N=900):
```



(b) We see that the distance increases quickly, then stabilizes. The N=9000 network has a higher average distance, which makes sense since larger graphs have larger diameters

The variance flucuates but stays relatively stable. The larger graph has higher variance, probably because a larger graph has more possible connectivity paths.

(c) The degree distribution follows a binomial like shape which is expected in ER networks. The visited nodes have higher degrees than the full graph, which makes sense, since a random walker is more likely to visit nodes that have connections

(d) The larger network has a larger diameter, thus the walker can walk further before the average distance stabilizes. Variance is also higher because there are more possible paths to traverse. The diameter of the network plays a role, in that the nodes are further apart, so the walker takes longer to explore

Q 2.2 Random walk on networks with fat-tailed degree distribution

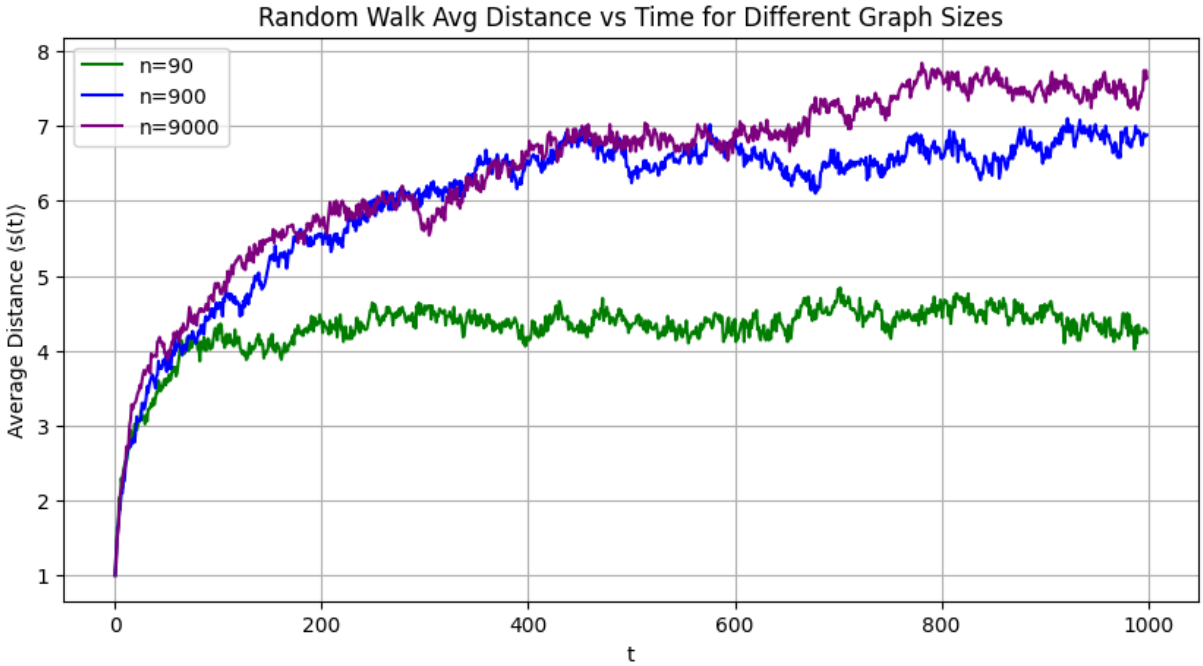
- (a) Generate an undirected preferential attachment network with 900 nodes, where each new node attaches to m = 1 old nodes.
- (b) Let a random walker start from a randomly selected node. Measure and plot ⟨s(t)⟩ v.s. t and σ2(t) v.s. t.

- (c) Measure the degree distribution of the nodes reached at the end of the random walk on this network. How does it compare with the degree distribution of the graph?
- (d) Repeat 2(b) for preferential attachment networks with 90 and 9000 nodes, and m = 1. Compare the results and explain qualitatively. Does the diameter of the network play a role?

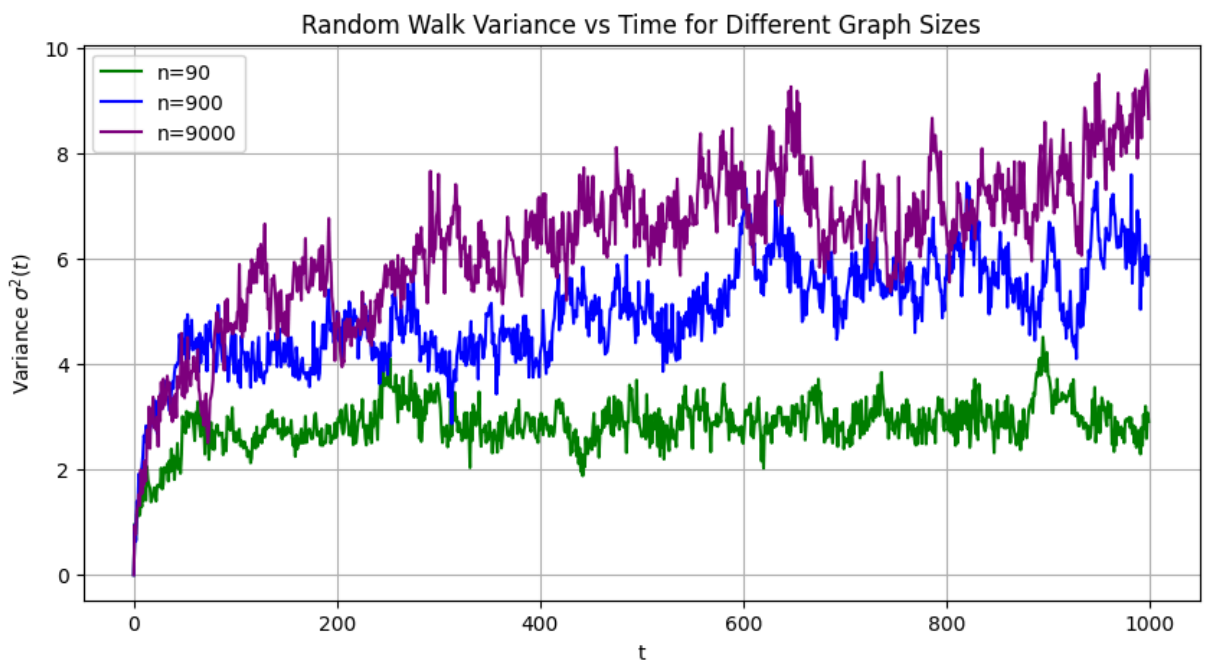
In [84]:
```
def generate_pa_graph(n=900, m=1):
    return ig.Graph.Barabasi(n=n, m=m, directed=False)

# 2.2(b)
def random_walk_distance_stats(g, steps=1000, trials=100):
    dist_matrix = g.shortest_paths()
    n = g.vcount()

    sum_dist = np.zeros(steps)
    sum_sq_dist = np.zeros(steps)
    count = np.zeros(steps)

    for _ in range(trials):
        start = random.randint(0, n - 1)
        pos = start
        for t in range(steps):
            neighbors = g.neighbors(pos)
            if not neighbors:
                break
            pos = random.choice(neighbors)
            dist = dist_matrix[start][pos]

            sum_dist[t] += dist
            sum_sq_dist[t] += dist**2
            count[t] += 1

    avg_dist = sum_dist / count
    var_dist = sum_sq_dist / count - avg_dist**2

    return avg_dist, var_dist

# 2.2(c)
def degree_distribution_of_end_nodes(g, steps=100, trials=1000):
    degrees = []
    for _ in range(trials):
        pos = random.randint(0, g.vcount() - 1)
        for _ in range(steps):
            neighbors = g.neighbors(pos)
            if not neighbors:
                break
            pos = random.choice(neighbors)
        degrees.append(g.degree(pos))
    return degrees

# 2.2(d)
def run_experiments():
    # generate random seeed:
    np.random.seed(42)
```

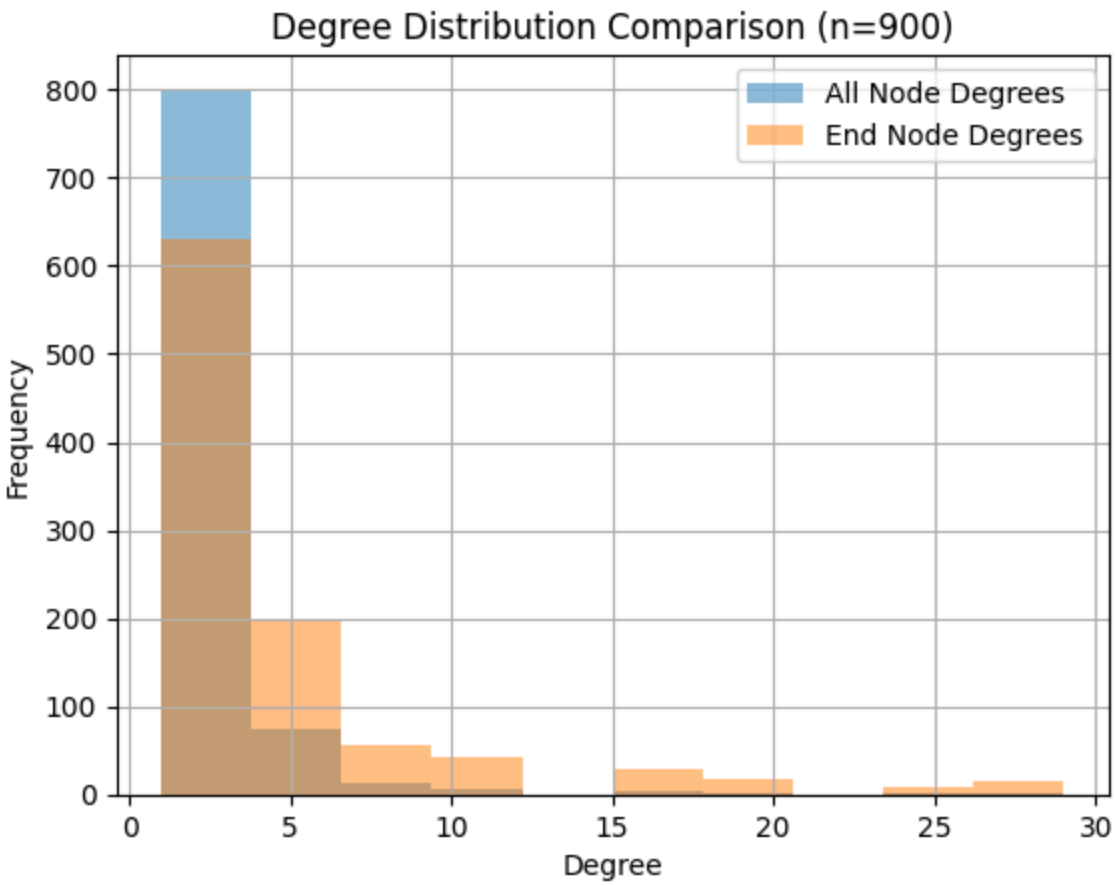
```python
    random.seed(42)

    sizes = [90, 900, 9000]
    colors = ['green', 'blue', 'purple']
    plt.figure(figsize=(10, 5))

    for size, color in zip(sizes, colors):
        g = generate_pa_graph(n=size, m=1)
        diam = g.diameter(directed=False)
        avg_dist, _ = random_walk_distance_stats(g)
        plt.plot(avg_dist, label=f'n={size}', color=color)

    plt.xlabel("t")
    plt.ylabel("Average Distance ⟨s(t)⟩")
    plt.title("Random Walk Avg Distance vs Time for Different Graph Sizes")
    plt.legend()
    plt.grid(True)
    plt.show()

    plt.figure(figsize=(10, 5))
    for size, color in zip(sizes, colors):
        g = generate_pa_graph(n=size, m=1)
        diam = g.diameter(directed=False)
        print(f"Graph size n={size}, diameter = {diam}")
        _, var_dist = random_walk_distance_stats(g)
        plt.plot(var_dist, label=f'n={size}', color=color)

    plt.xlabel("t")
    plt.ylabel("Variance $\sigma^2(t)$")
    plt.title("Random Walk Variance vs Time for Different Graph Sizes")
    plt.legend()
    plt.grid(True)
    plt.show()


    # Degree comparison for n=900
    g = generate_pa_graph(n=900)
    walk_degrees = degree_distribution_of_end_nodes(g)
    all_degrees = g.degree()

    plt.hist(all_degrees, bins=10, alpha=0.5, label='All Node Degrees')
    plt.hist(walk_degrees, bins=10, alpha=0.5, label='End Node Degrees')
    plt.xlabel("Degree")
    plt.ylabel("Frequency")
    plt.legend()
    plt.title("Degree Distribution Comparison (n=900)")
    plt.grid(True)
    plt.show()

run_experiments()
```

```
C:\Users\Anwesha\AppData\Local\Temp\ipykernel_71488\984534709.py:6: DeprecationWarni
ng: Graph.shortest_paths() is deprecated; use Graph.distances() instead
  dist_matrix = g.shortest_paths()
```




```
Graph size n=90, diameter = 9
Graph size n=900, diameter = 19
Graph size n=9000, diameter = 31
```





(a) above

(b) above

(c) While the overall node degree distribution and the distribution of end nodes follows a power-lawa, they are not identical. Random walks on preferential attachment networks are more likely to end up on higher degree nodes, even if most nodes of the graph have a lower degree. This is because the higher degree nodes have a larger number of connections, which makes it more likely that a random walker will be able to get to them, while walking randomly. Thus we see higher degree nodes being overepresented in the end-node distribution when compared to the overal degree distribution

(d) as the network size incresase the rate of growth for the average distance and the variance is slower, and they reach higher saturation values. This is because the larger the network is, the more paths there are and the more nodes there are to explore. Thus a larger diameter means the random walk takes longer to cover the graph. Thus the diameter plays a major role in shaping the growth

Q2.3 PageRank

The PageRank algorithm, as used by the Google search engine, exploits the linkage structure of the web to compute global “importance” scores that can be used to influence the ranking

of search results. Here, we use random walk to simulate PageRank.

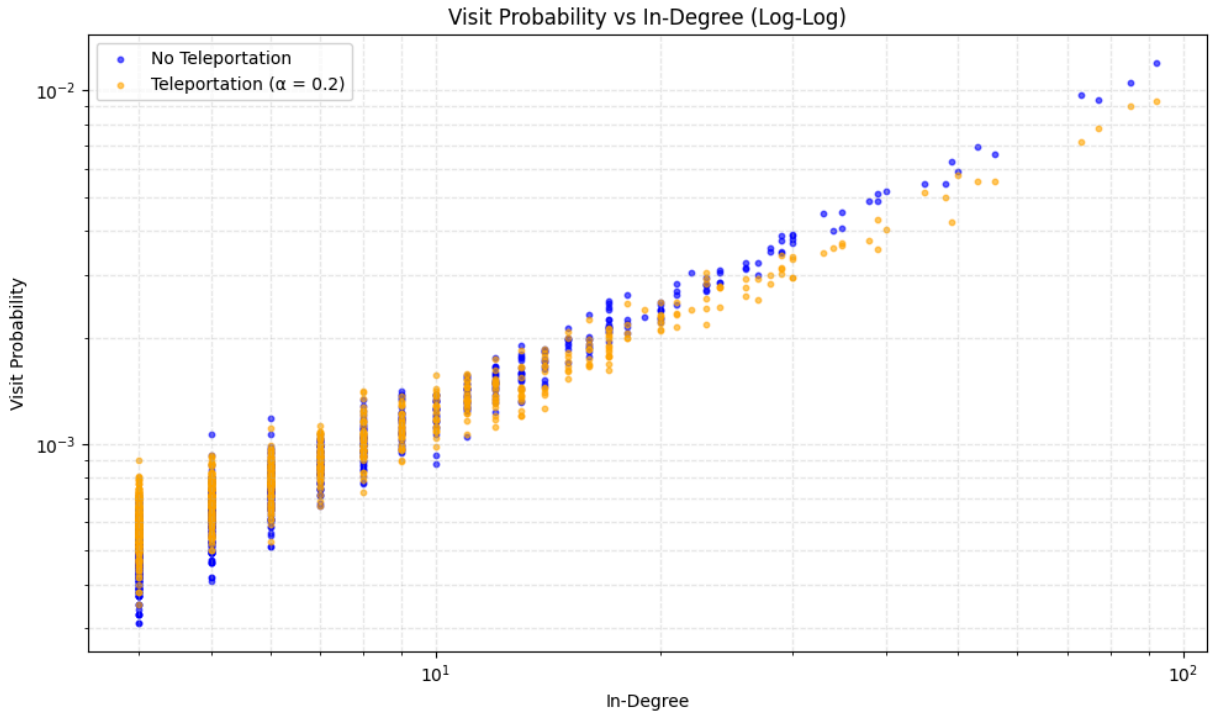
(a) We are going to create a directed random network with 900 nodes, using the preferential attachment model. Note that in a directed preferential attachment network, the out-degree of every node is m, while the in-degrees follow a power law distribution. One problem of performing random walk in such a network is that, the very first node will have no outbounding edges, and be a “black hole” which a random walker can never “escape” from. To address that, let’s generate another 900-node random network with preferential attachment model, and merge the two networks by adding the edges of the second graph to the first graph with a shuffling of the indices of the nodes.

Create such a network using m = 4. Measure the probability that the walker visits each node. Is this probability related to the degree of the nodes?

(b) In all previous questions, we didn’t have any teleportation. Now, we use a teleportation probability of α = 0.2 (teleport out of a node with prob=0.2 instead of going to its neighbor). By performing random walks on the network created in 3(a), measure the probability that the walker visits each node. How is this probability related to the degree of the node and α ?

In [85]:
```
import igraph as ig
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import Counter

# Parameters
N = 1000                # Number of nodes in the preferential attachment graph
m = 4                   # Each new node attaches to m existing nodes
steps = 100000          # Total steps for the random walk
alpha = 0.2             # Teleportation probability

def create_preferential_attachment_graph(N, m):
    """Creates a directed preferential attachment graph."""
    # Generate undirected BA graph first, then make it directed both ways
    return ig.Graph.Barabasi(n=N, m=m, directed=False).as_directed()

def random_walk(G, steps=100000, teleport=False, alpha=0.2):
    """Performs a random walk on graph G, optionally with teleportation."""
    visit_count = Counter()
    nodes = list(range(G.vcount()))
    current = random.choice(nodes)

    for _ in range(steps):
        visit_count[current] += 1

        if teleport and random.random() < alpha:
            current = random.choice(nodes)
        else:
            neighbors = G.successors(current)
            if neighbors:
```

```
                current = random.choice(neighbors)
            else:
                # If no outgoing edges, teleport/restart
                current = random.choice(nodes)

    total_visits = sum(visit_count.values())
    visit_prob = {node: count / total_visits for node, count in visit_count.items()
    return visit_prob

def plot_visit_vs_indegree(G, visit_prob, label, color):
    """Plots visit probability vs in-degree on a log-log scale."""
    in_degrees = G.degree(mode="in")
    x = []
    y = []

    for node, prob in visit_prob.items():
        deg = in_degrees[node]
        x.append(deg)
        y.append(prob)

    plt.scatter(x, y, label=label, alpha=0.6, s=10, color=color)

# Create graph
G = create_preferential_attachment_graph(N, m)

# Perform random walk without teleportation
visit_prob_no_teleport = random_walk(G, steps, teleport=False)

# Perform random walk with teleportation
visit_prob_teleport = random_walk(G, steps, teleport=True, alpha=alpha)

# Plotting
plt.figure(figsize=(10, 6))
plot_visit_vs_indegree(G, visit_prob_no_teleport, label="No Teleportation", color='
plot_visit_vs_indegree(G, visit_prob_teleport, label="Teleportation (α = 0.2)", col

plt.xscale("log")
plt.yscale("log")
plt.xlabel("In-Degree")
plt.ylabel("Visit Probability")
plt.title("Visit Probability vs In-Degree (Log-Log)")
plt.legend()
plt.grid(True, which="both", ls="--", alpha=0.3)
plt.tight_layout()
plt.show()
```



(a) we see that the probability of a node being visited is strongly correlated with its in degree. The higher in-degree nodes are visited a lot more than lower in-degree nodes. The log-log plot shows a power-law relationship between visit probability and in-degree

(b) When there is a 20% teleportation probability, the walker jumps to a random node than following outgoing edges. Thus nodes with lower indegrees get more visits than before, since teleportation allows visting them via random jumps. This has the effect of making the power law relationship between the probability and the in-degree of the nodes less steep.

Q 2.4 Personalized PageRank

While the use of PageRank has proven very effective, the web's rapid growth in size and diversity drives an increasing demand for greater flexibility in ranking. Ideally, each user should be able to define their own notion of importance for each individual query.

(a) Suppose you have your own notion of importance. Your interest in a node is proportional to the node's PageRank, because you totally rely upon Google to decide which website to visit (assume that these nodes represent websites). Again, use random walk on network generated in question 3 to simulate this personalized PageRank. Here the teleportation probability to each node is proportional to its PageRank (as opposed to the regular PageRank, where at teleportation, the chance of visiting all nodes are the same and equal to 1/N). Again, let the teleportation probability be equal to α = 0.2. Compare the results with 3(a).

(b) Find two nodes in the network with median PageRanks. Repeat part 4(a) if teleportations land only on those two nodes (with probabilities 1/2, 1/2). How are the PageRank values

affected?

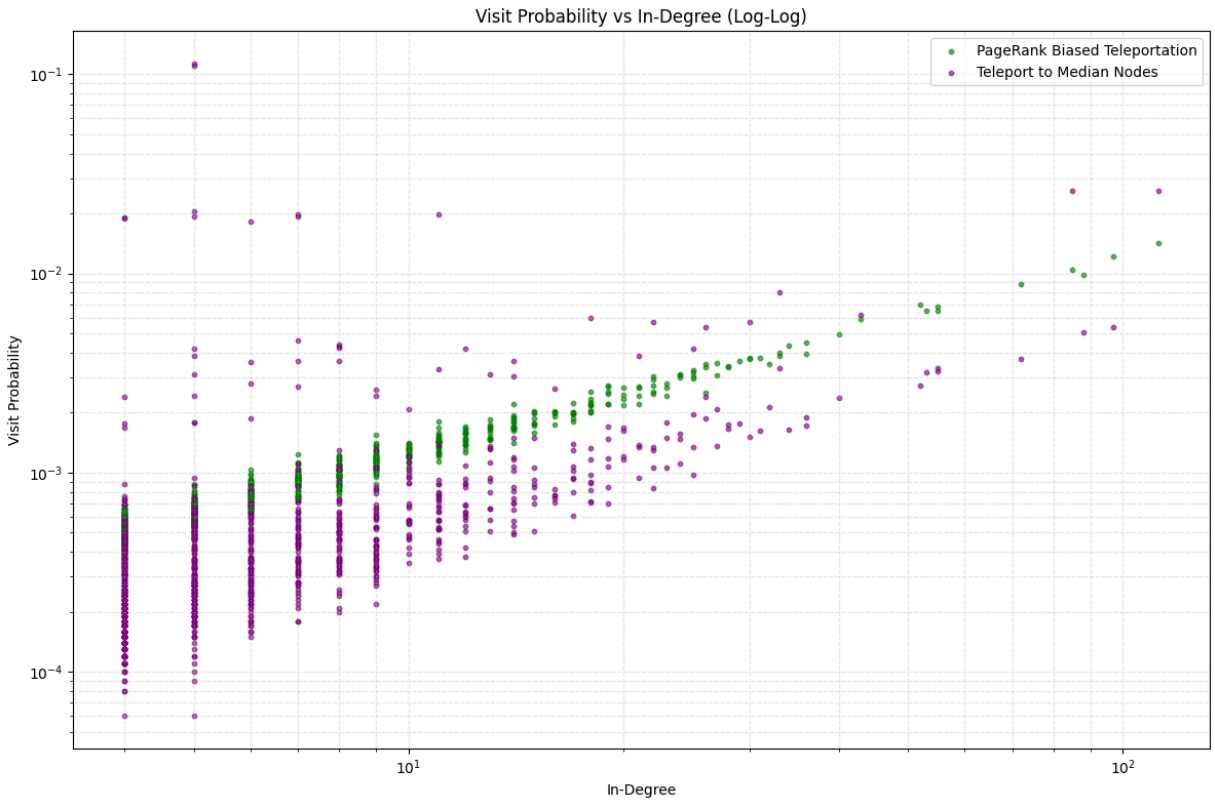
(c) More or less, 4(b) is what happens in the real world, in that a user browsing the web only teleports to a set of trusted web pages. However, this is against the assumption of normal PageRank, where we assume that people's interest in all nodes are the same. Can you take into account the effect of this self-reinforcement and adjust the PageRank equation?

In [86]:
```
import igraph as ig
import numpy as np
import matplotlib.pyplot as plt
import random
from collections import Counter

# Parameters
N = 1000                # Number of nodes in the preferential attachment graph
m = 4                   # Each new node attaches to m existing nodes
steps = 100000          # Total steps for the random walk
alpha = 0.2             # Teleportation probability

def create_preferential_attachment_graph(N, m):
    """Creates a directed preferential attachment graph."""
    return ig.Graph.Barabasi(n=N, m=m, directed=False).as_directed()

def random_walk(
    G,
    steps=100000,
    teleport=False,
    alpha=0.2,
    teleport_nodes=None,
    teleport_probs=None,
    pagerank_bias=False,
    pagerank=None
):
    """Performs a random walk on graph G, optionally with teleportation."""
    visit_count = Counter()
    nodes = list(range(G.vcount()))
    current = random.choice(nodes)

    for _ in range(steps):
        visit_count[current] += 1

        if teleport and random.random() < alpha:
            if pagerank_bias:
                # Teleportation probability proportional to PageRank
                current = random.choices(nodes, weights=pagerank, k=1)[0]
            elif teleport_nodes is not None:
                # Teleport only to specified nodes
                current = random.choices(teleport_nodes, weights=teleport_probs, k=
            else:
                # Uniform teleportation
                current = random.choice(nodes)
        else:
            neighbors = G.successors(current)
            if neighbors:
```

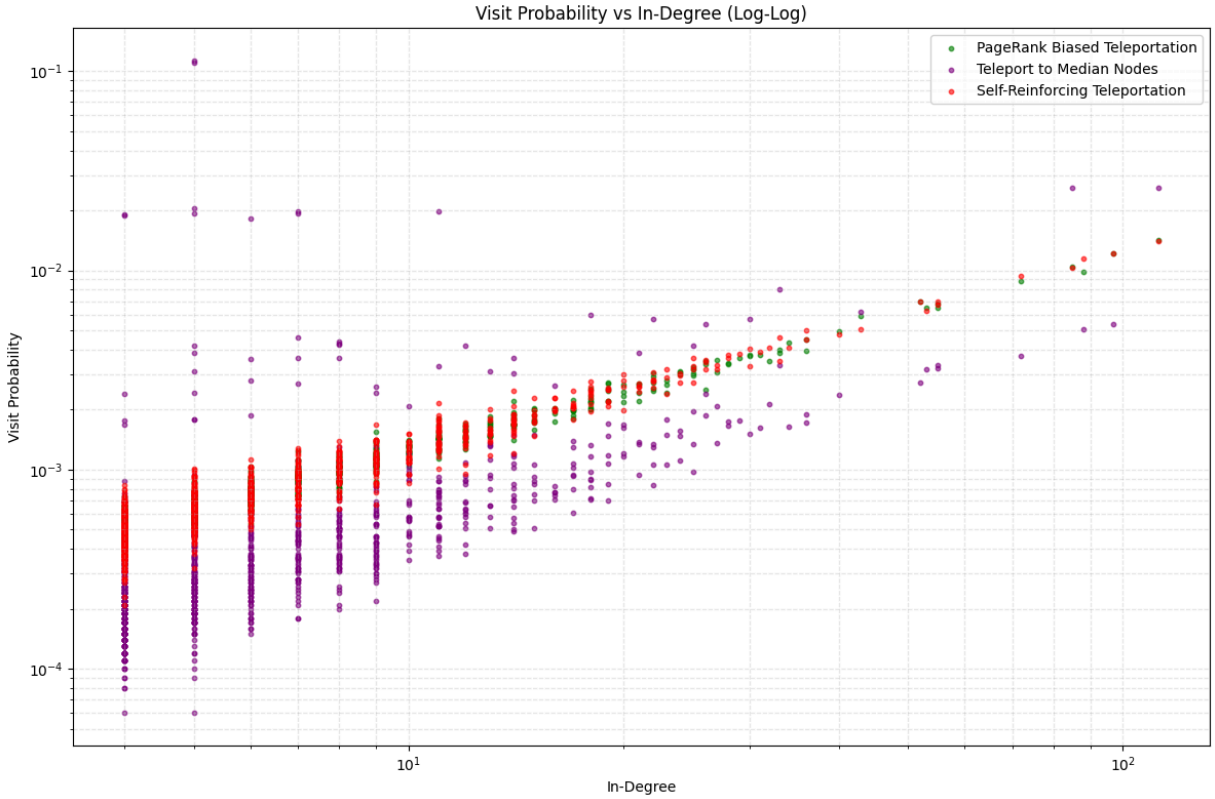
```
                current = random.choice(neighbors)
            else:
                # If no outgoing edges, teleport or restart
                current = random.choice(nodes)

    total_visits = sum(visit_count.values())
    visit_prob = {node: count / total_visits for node, count in visit_count.items()
    return visit_prob

def plot_visit_vs_indegree(G, visit_prob, label, color):
    """Plots visit probability vs in-degree on a log-log scale."""
    in_degrees = G.degree(mode="in")
    x = []
    y = []

    for node, prob in visit_prob.items():
        deg = in_degrees[node]
        x.append(deg)
        y.append(prob)

    plt.scatter(x, y, label=label, alpha=0.6, s=10, color=color)

# Create graph
G = create_preferential_attachment_graph(N, m)

# Compute PageRank
pagerank_values = G.pagerank()
pagerank_dict = {i: pagerank_values[i] for i in range(G.vcount())}

# Perform random walk with PageRank-biased teleportation
visit_prob_pagerank_bias = random_walk(
    G,
    steps,
    teleport=True,
    alpha=alpha,
    pagerank_bias=True,
    pagerank=pagerank_values
)

# Find two nodes with median PageRanks
median_pagerank = np.median(pagerank_values)
median_nodes = [node for node, rank in pagerank_dict.items() if abs(rank - median_p

if len(median_nodes) < 2:
    # Find the two nodes closest to the median PageRank
    sorted_nodes = sorted(pagerank_dict.items(), key=lambda x: abs(x[1] - median_pa
    median_nodes = [sorted_nodes[0][0], sorted_nodes[1][0]]
else:
    median_nodes = median_nodes[:2]

# Perform random walk with teleportation to median nodes
teleport_probs = [0.5, 0.5]
visit_prob_median_teleport = random_walk(
    G,
    steps,
    teleport=True,
```

```
    alpha=alpha,
    teleport_nodes=median_nodes,
    teleport_probs=teleport_probs
)

# Plotting
plt.figure(figsize=(12, 8))
plot_visit_vs_indegree(G, visit_prob_pagerank_bias, label="PageRank Biased Teleport
plot_visit_vs_indegree(G, visit_prob_median_teleport, label="Teleport to Median Nod

plt.xscale("log")
plt.yscale("log")
plt.xlabel("In-Degree")
plt.ylabel("Visit Probability")
plt.title("Visit Probability vs In-Degree (Log-Log)")
plt.legend()
plt.grid(True, which="both", ls="--", alpha=0.3)
plt.tight_layout()
plt.show()

# Print PageRank values for the two median nodes
print(f"PageRank of median node 1: {pagerank_dict[median_nodes[0]]}")
print(f"PageRank of median node 2: {pagerank_dict[median_nodes[1]]}")

def self_reinforcing_random_walk(G, steps=100000, alpha=0.2):
    """Performs a self-reinforcing random walk where teleportation targets are base
    visit_count = Counter()
    nodes = list(range(G.vcount()))
    current = random.choice(nodes)
    epsilon = 1e-6  # To avoid division by zero early on

    for _ in range(steps):
        visit_count[current] += 1

        if random.random() < alpha:
            # Self-reinforcing teleportation: more visited nodes are more likely to
            visit_weights = [visit_count[n] + epsilon for n in nodes]
            current = random.choices(nodes, weights=visit_weights, k=1)[0]
        else:
            neighbors = G.successors(current)
            if neighbors:
                current = random.choice(neighbors)
            else:
                current = random.choice(nodes)

    total_visits = sum(visit_count.values())
    visit_prob = {node: count / total_visits for node, count in visit_count.items()
    return visit_prob

# Run the self-reinforcing version
visit_prob_self_reinforcing = self_reinforcing_random_walk(G, steps, alpha=alpha)

# Plot
plt.figure(figsize=(12, 8))
plot_visit_vs_indegree(G, visit_prob_pagerank_bias, label="PageRank Biased Teleport
plot_visit_vs_indegree(G, visit_prob_median_teleport, label="Teleport to Median Nod
```

```
plot_visit_vs_indegree(G, visit_prob_self_reinforcing, label="Self-Reinforcing Tele

plt.xscale("log")
plt.yscale("log")
plt.xlabel("In-Degree")
plt.ylabel("Visit Probability")
plt.title("Visit Probability vs In-Degree (Log-Log)")
plt.legend()
plt.grid(True, which="both", ls="--", alpha=0.3)
plt.tight_layout()
plt.show()
```




```
PageRank of median node 1: 0.0007010148927129625
PageRank of median node 2: 0.0007014204108930892
```



(a) Comparing with 3(a) we see that the probability of visting a node remains positively correlated with its in-degree. When compared to the orange points in 3(a) the green node sin 4(a) seem to have a higher visit probability. Thus the PageRank by Google helps visit nodes higher in visit probability

(b) For 4(b), focusing teleportation on the two nodes that initially had median PageRank values around 0.0007 produces these effects:

Visit probability for those two nodes dramatically increases, shown by the single purple dot at approximately 10^-1 on the graph.

The correlation between in-degree and visit probability weakens substantially. The purple data points appear scattered, showing that in-degree is no longer a reliable predictor.

Some low in-degree nodes show higher visit probabilities than expected, likely due to network proximity to one of the median nodes.

The inherent link structure of the network is overshadowed, resulting in the skewed importance of the teleported median nodes.

(c) In real-world browsing , users tend to revisit popular sites or sites they have visited before, creating a self-reinforcing effect. This is modelled by the teleportation behavior, where the probablity of jumping to node increases with its past visit count.

This causes the already popular nodes to become even more dominant, making the visit probability becoming more concentrated around the high in-degree nodes.

This makes the ranking system more adaptive to user behavior but also introduces bia by reinforing early popularity